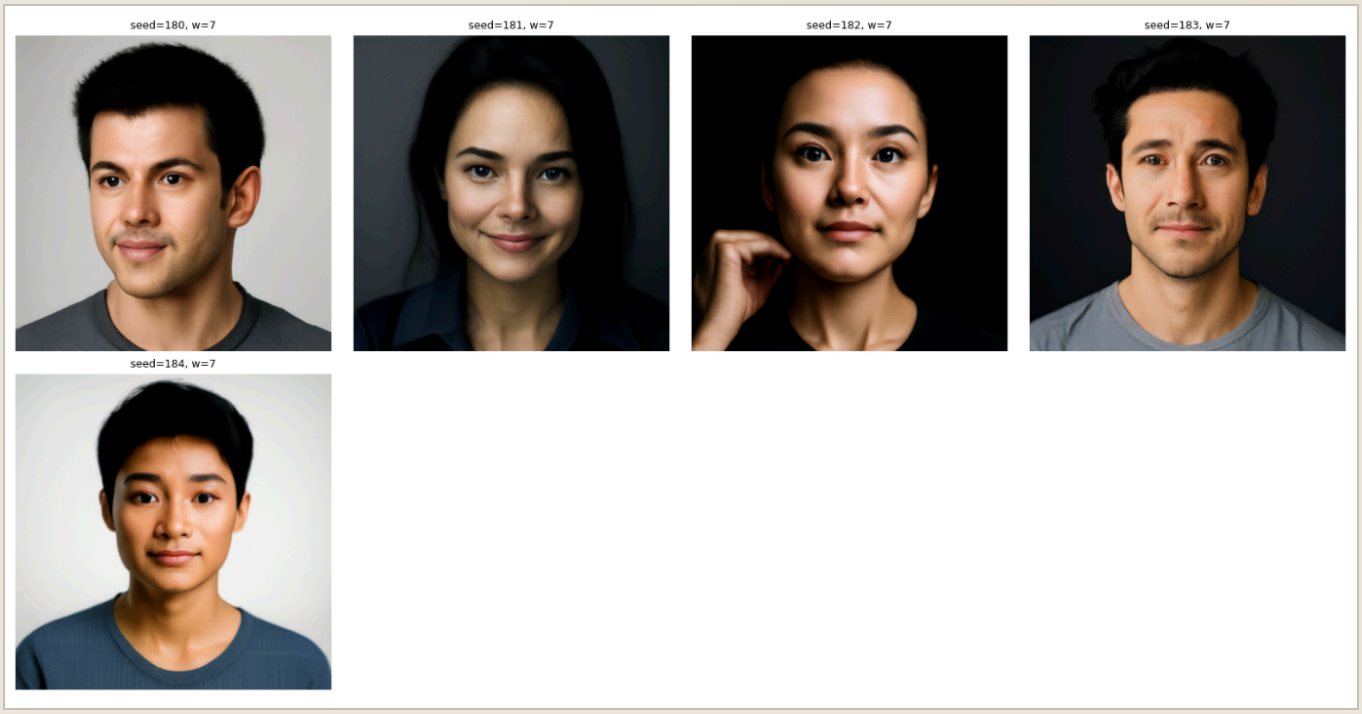
Project 3 · Part A

Exploring the Power of Flow Models

Noising, denoising, Euler sampling, classifier-free guidance, and image editing with a pretrained flow-matching model.



Five samples of “a high quality photo” generated from pure noise with classifier-free guidance ($w = 7$).

Setup and Conventions

In this part of the project I used a pretrained text-to-image flow model, PixNerd-XXL-P16-T2I, which generates 512×512 images. I did not train anything myself. The model can't read a raw text string, so each prompt is first turned into an embedding with `model.encode(prompt)`, which uses Qwen3. Instead, I wrote the pieces that sit around the model: building noisy inputs, turning the model's velocity prediction into a clean image, integrating with Euler steps, classifier-free guidance, and image-to-image editing.

Throughout, I used the linear flow-matching convention, where time 0 is pure noise and time 1 is a clean image:

$$x_t = (1 - t)\,\epsilon + t\,x_1, \qquad \epsilon \sim \mathcal{N}(0, I)$$

Differentiating with respect to t gives the velocity along this path, $v = x_1 - \epsilon$, which is what the model $v_\theta(x_t, t, c)$ is trained to predict. All images are center-cropped and resized to 512×512 with RGB values in $[-1, 1]$.

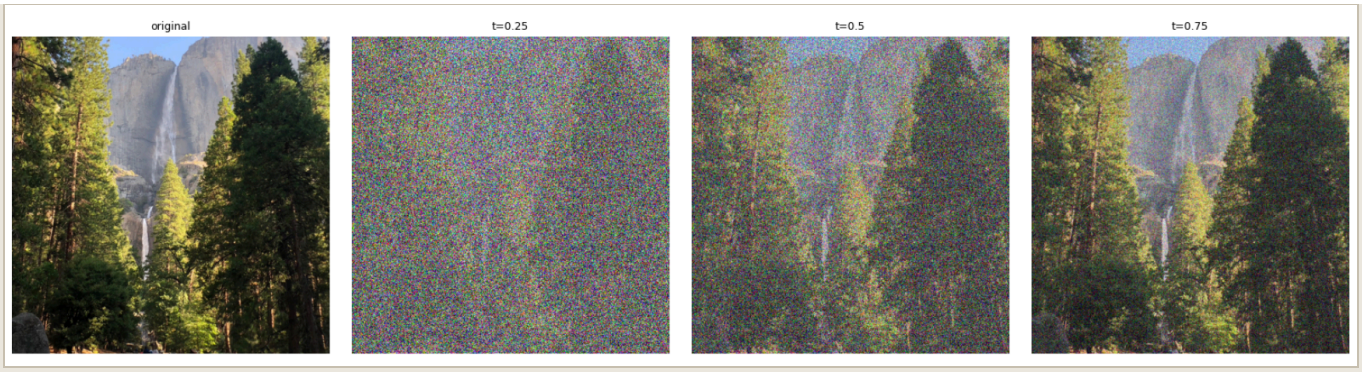
Setting	Value
Random seed	180
Euler step size h	0.02
CFG weight w	7
Default prompt	“a high quality photo”
Null prompt (for CFG)	“” (empty string)

Part 1.1

Constructing Noisy Training Inputs

A flow model is trained to predict the velocity at time t starting from noisy images. We set `t=0` to a noisy image and `t=1` to a clean image. For this task, I interpolated the manifold between these two endpoints by blending the noisy image and the clean image, and represented each image as `(1 - t) * noise + t * clean`. I checked both endpoints so it returns both exactly the noise and exactly the image.

My original photo of Yosemite Falls.



Left to right: preprocessed original, then x_t at $t = 0.25, 0.50, 0.75$. Lower t means more noise.

I sampled a single noise tensor (seed 180) and reused it for $t = 0.25, 0.5, 0.75$.

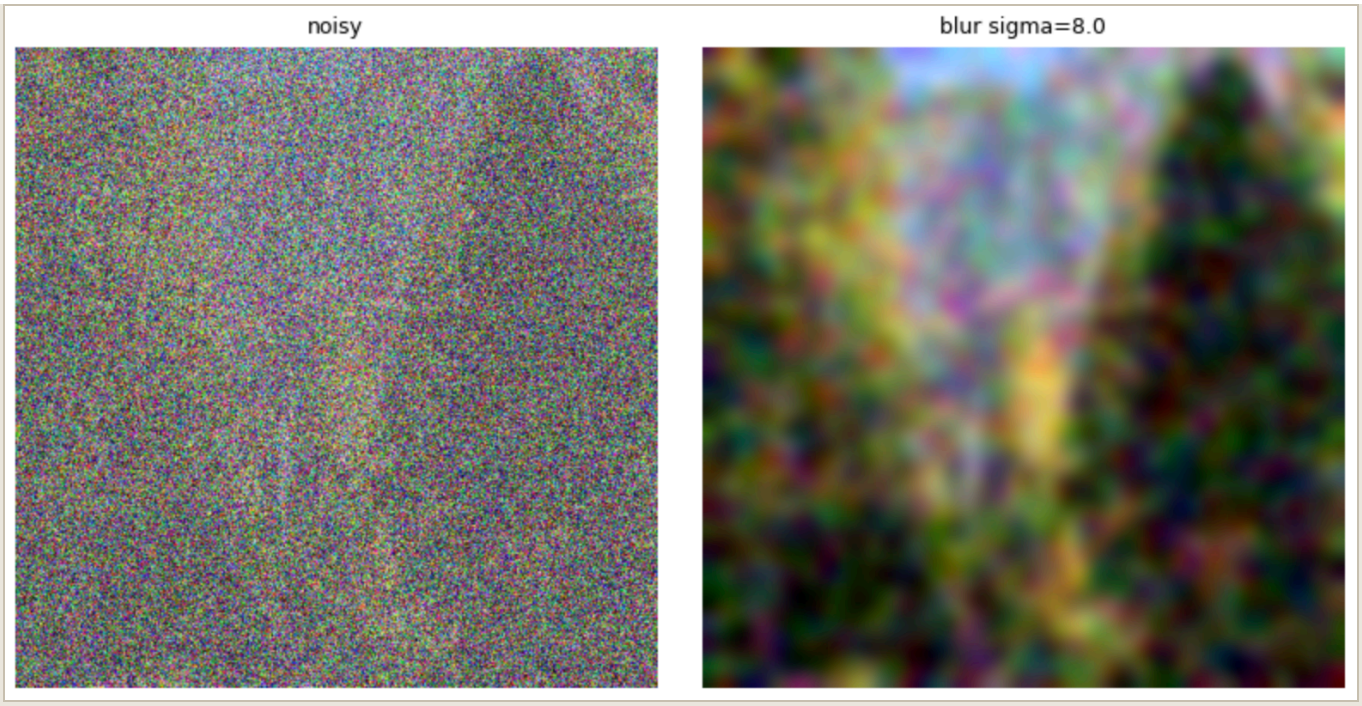
At $t = 0.25$ the image is mostly noise, and I can only just make out the dark tree on the right. By $t = 0.75$ the scene is clearly visible with a layer of grain on top.

Part 1.2

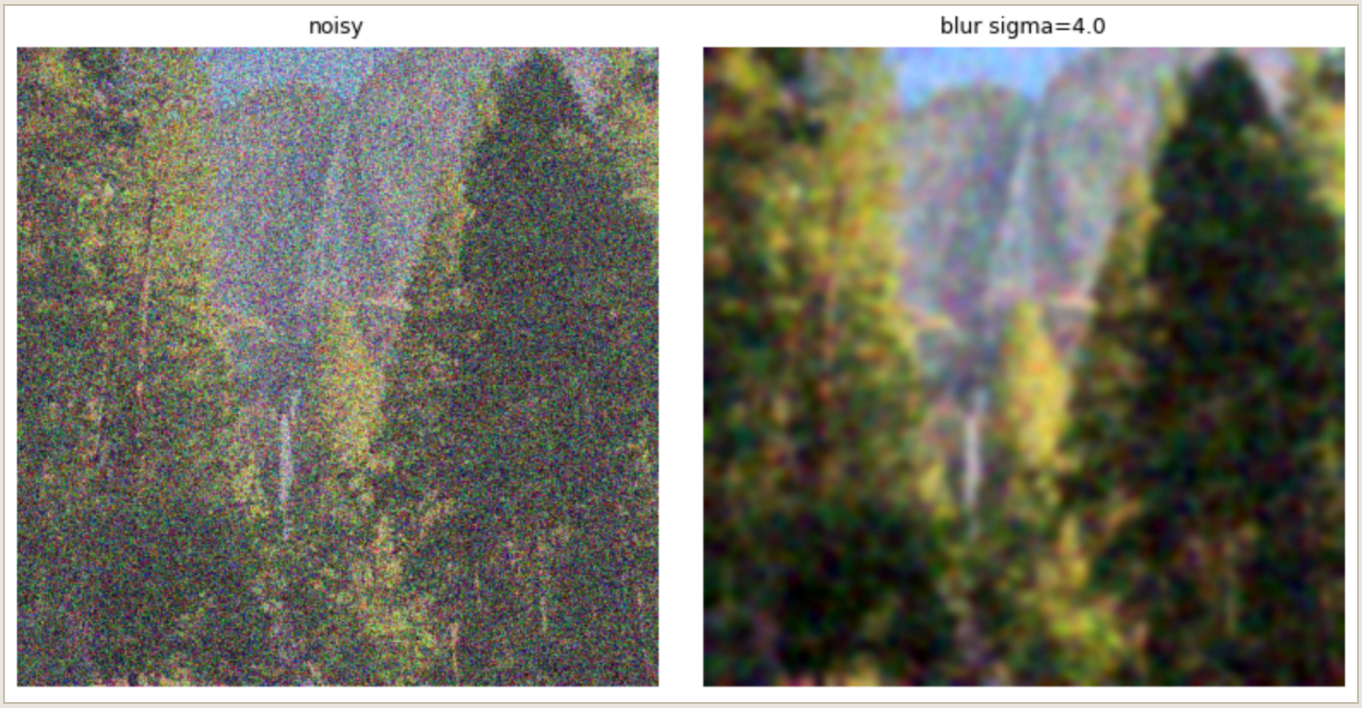
Classical Denoising

In this task, I tried to denoise each input without the model. Since x_t only contains t times the clean image, I first divided by t to restore the image amplitude, then applied a Gaussian blur: $\hat{x}_1 = G_\sigma * (x_t/t)$. Dividing by t only fixes the brightness and does not remove any noise. The blur does the denoising, because averaging nearby pixels partly cancels independent, zero-mean noise. I swept $\sigma \in \{1, 2, 4, 8, 16\}$ and kept the blur with the lowest mean squared error against the known clean image.

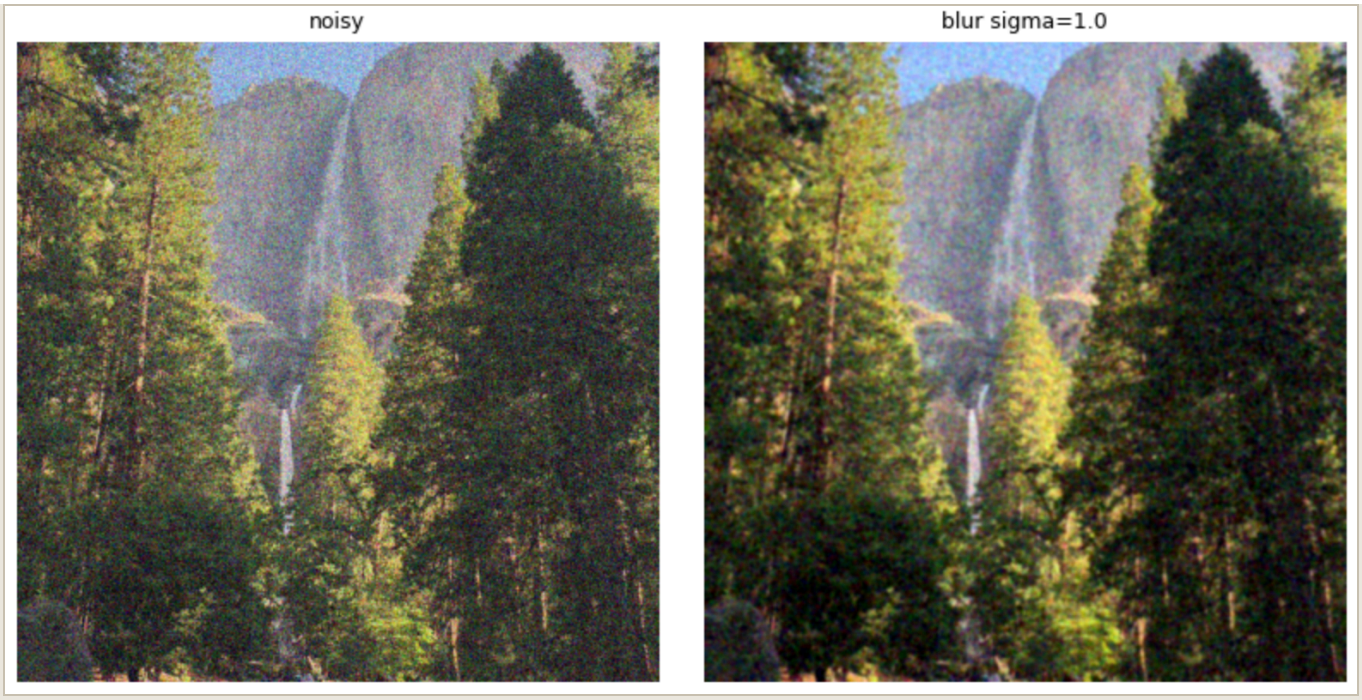
When you denoise the noisy input, we want to have larder σ. A larger blur averages away more noise but also removes the detail of the scene. At $t = 0.25$ the result is only a blurry outline of the trees and sky.



$t = 0.25$: noisy input (left) and best Gaussian blur, $\sigma = 8$ (right).



$t = 0.50$: noisy input (left) and best Gaussian blur, $\sigma = 4$ (right).



$t = 0.75$: noisy input (left) and best Gaussian blur, $\sigma = 1$ (right).

Part 1.3

One-Step Denoising

In this task, I implemented this in `one_step_denoise` that predicts a velocity, not noise, so the prediction is added (scaled by 1-t) rather than subtracted.

$$\hat{x}_1 = x_t + (1 - t)\, v_\theta(x_t, t, c)$$

If the model's velocity were exactly $x_1 - \epsilon$, then one jump of length $1 - t$ would land on the clean image.

Before using this model, I checked that the predicted velocity has the same shape, dtype and device as the image and contains only finite values.

Rows: $t = 0.25, 0.50, 0.75$. Columns: original, noisy input, one-step estimate.

This is much better than the Gaussian blur. At $t = 0.75$ and $t = 0.50$ the estimate is close to the original. At $t = 0.25$ the overall layout (trees, cliff, waterfall) is correct, but the result is soft and smooth. With that much noise, the model cannot tell which fine detail is correct, so one step predicts something close to the average of all plausible images, which looks blurry.

Part 1.4

Visualizing Velocity

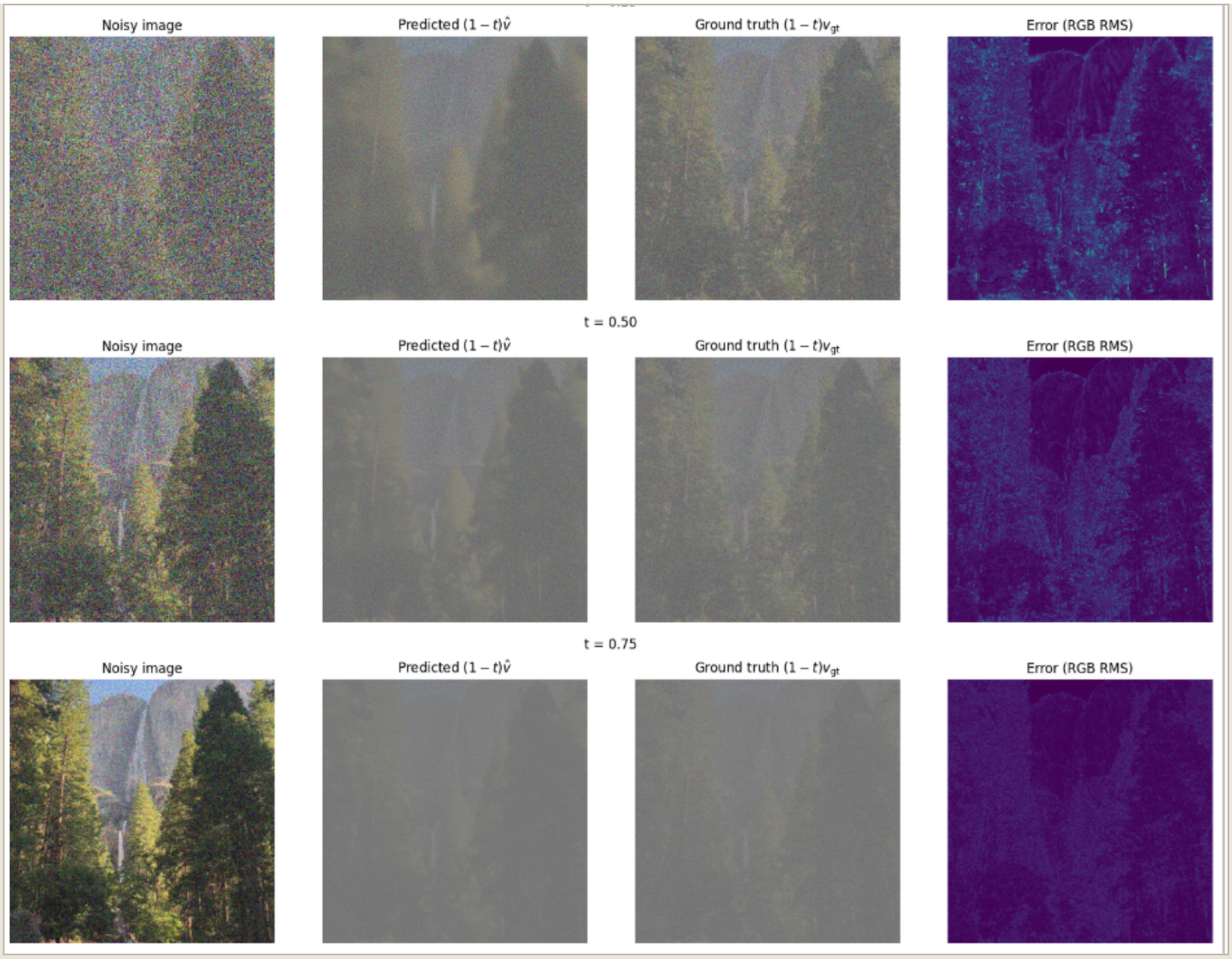
To see what the model adds in that one step, I compared the predicted update $(1-t)\hat{v}_t$ with the ground-truth update. Because I saved the noise, the ground-truth velocity is $v_{\text{gt}} = x_1 - \epsilon$, so $(1-t)v_{\text{gt}} = x_1 - x_t$. I used this value to recover my original photo.

For the error map I took the RMS difference across the RGB channels at each pixel:

$$E_t = \sqrt{\text{mean}_{\text{RGB}}\left([(1-t)(\hat{v}_t - v_{\text{gt}})]^2\right)}$$

The updates are drawn on one shared symmetric color scale, with zero shown as middle gray. The error maps also share one linear scale across all three times, so they can be compared directly.

Original image x_1, used for the ground-truth velocity.



Rows: $t = 0.25, 0.50, 0.75$. Columns: noisy input, predicted update $(1-t)\hat{v}$, ground-truth update $(1-t)v_{\text{gt}}$, per-pixel RGB RMS error.

The predicted and ground-truth updates look very similar at all three times. As t grows, both updates fade toward gray because the factor $(1-t)$ shrinks and less correction is needed. The error is largest at $t = 0.25$ and is concentrated in high-frequency regions such as the tree branches and the edges of the waterfall. The flat sky and the smooth shadowed areas have low error. The model gets the coarse structure right, and the fine texture is where it has to guess.

Iterative Denoising with Euler Sampling

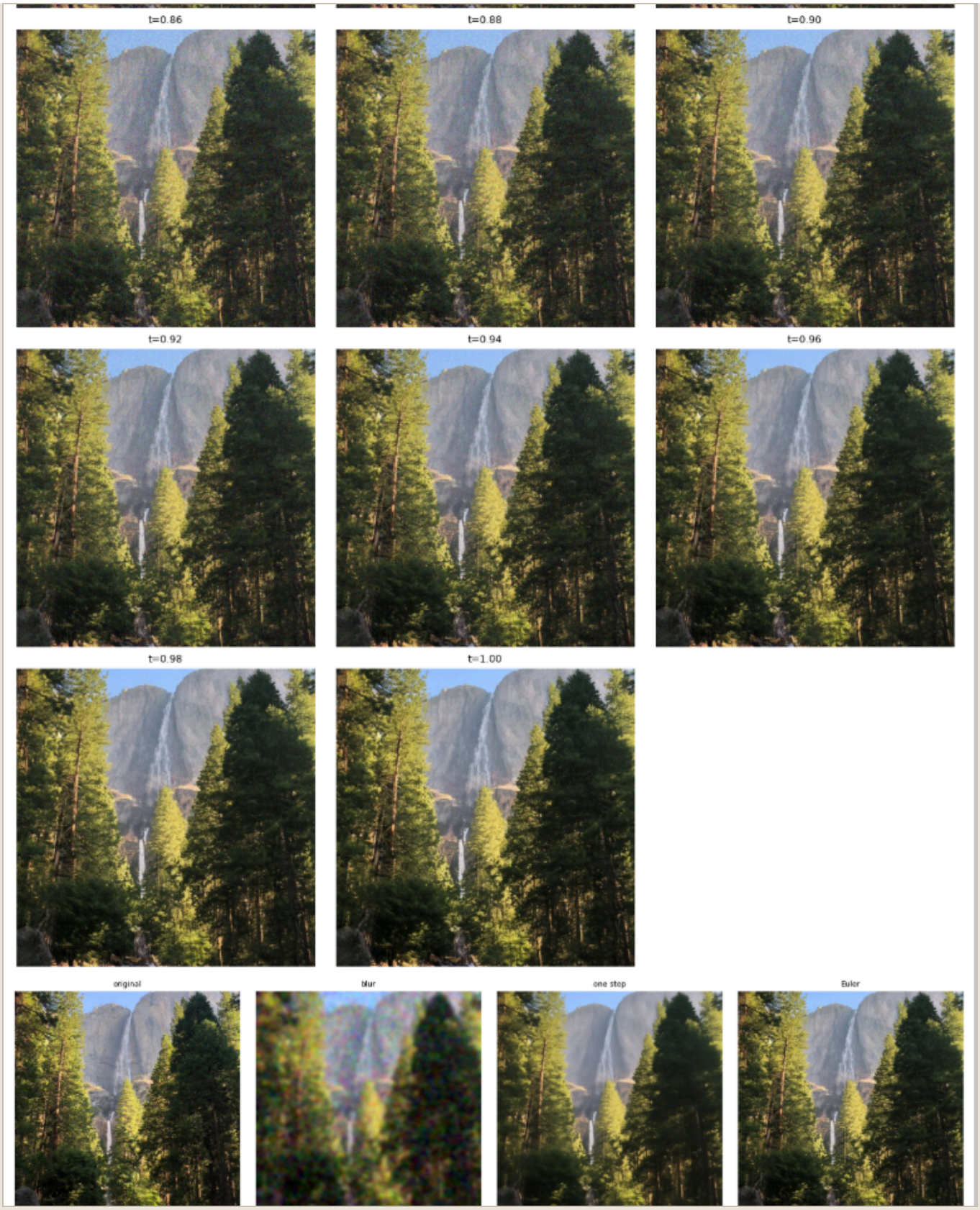
Instead of one big jump, `euler_sample` takes many small steps and asks the model for a new velocity at each one:

$$x_{t+h} = x_t + h \cdot v(x_t, t)$$

If the remaining time is shorter than h, the last step is shortened so the sampler lands exactly on $t = 1$. The function returns the final image and every (t, x_t) state along the way. I tested it with a constant velocity of 1: starting from 0, 0.5 and 0.93 it made 50, 25 and 4 calls and ended at exactly $1 - t_{\text{start}}$, which confirms the step count and the shortened final step are both correct.

I then denoised the saved $t = 0.50$ input with $h = 0.02$, which takes 25 updates.

t=0.50
t=0.52
t=0.54
t=0.56
t=0.58
t=0.60
t=0.62
t=0.64
t=0.66
t=0.68
t=0.70
t=0.72



Euler trajectory up to $t = 1.00$. Bottom row: original, Gaussian blur, one-step, Euler.

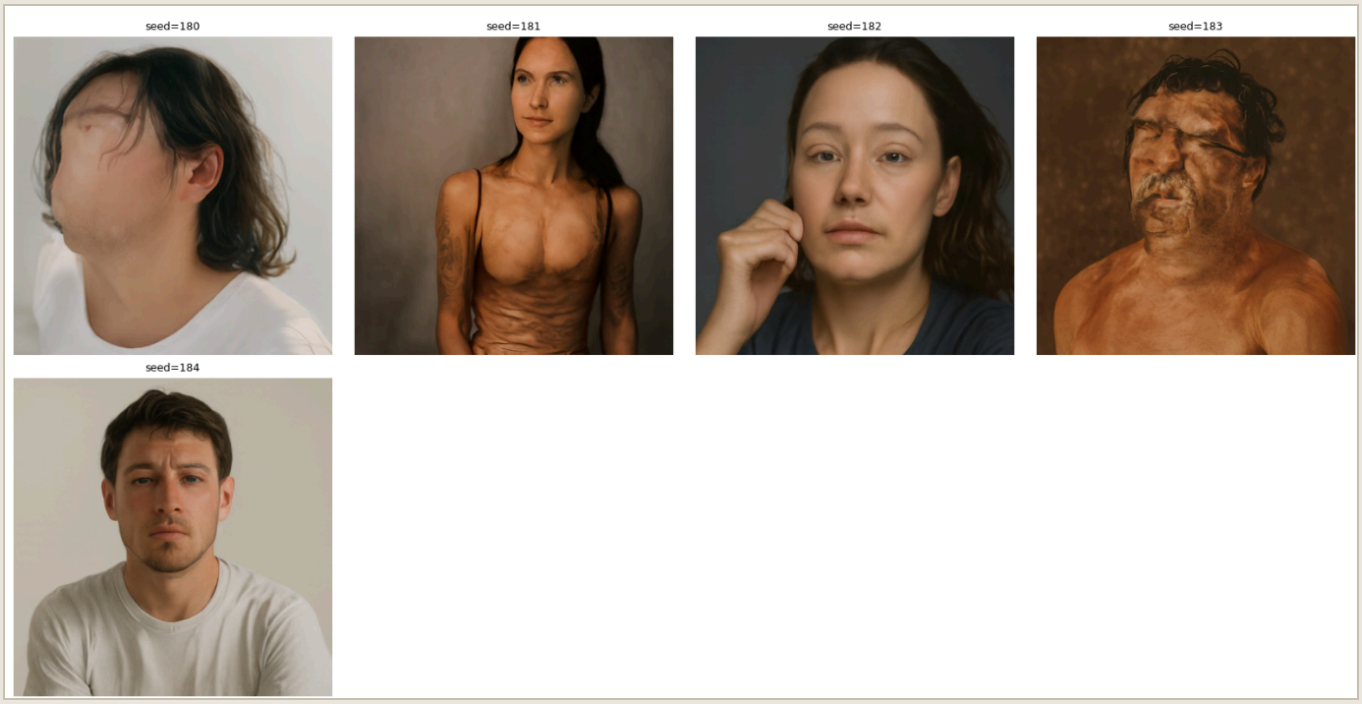
The noise disappears gradually as t becomes larger, and the final image is the sharpest of the three methods. The Gaussian blur removes noise by removing detail.

The one-step estimate keeps the structure but is a little soft. The Euler result recovers crisp needles on the trees and texture on the granite. Because the velocity is recomputed at each step, the model can correct its earlier guesses as the image gets cleaner, rather than committing to a single average guess.

Part 1.6

Generating Images from Noise

Starting Euler sampling at $t = 0$ from pure Gaussian noise turns the denoiser into a generator. I drew five independent noise tensors (seeds 180 to 184) and integrated each for 50 steps with the prompt “a high quality photo”.



Five samples from pure noise, no guidance, seeds 180–184.

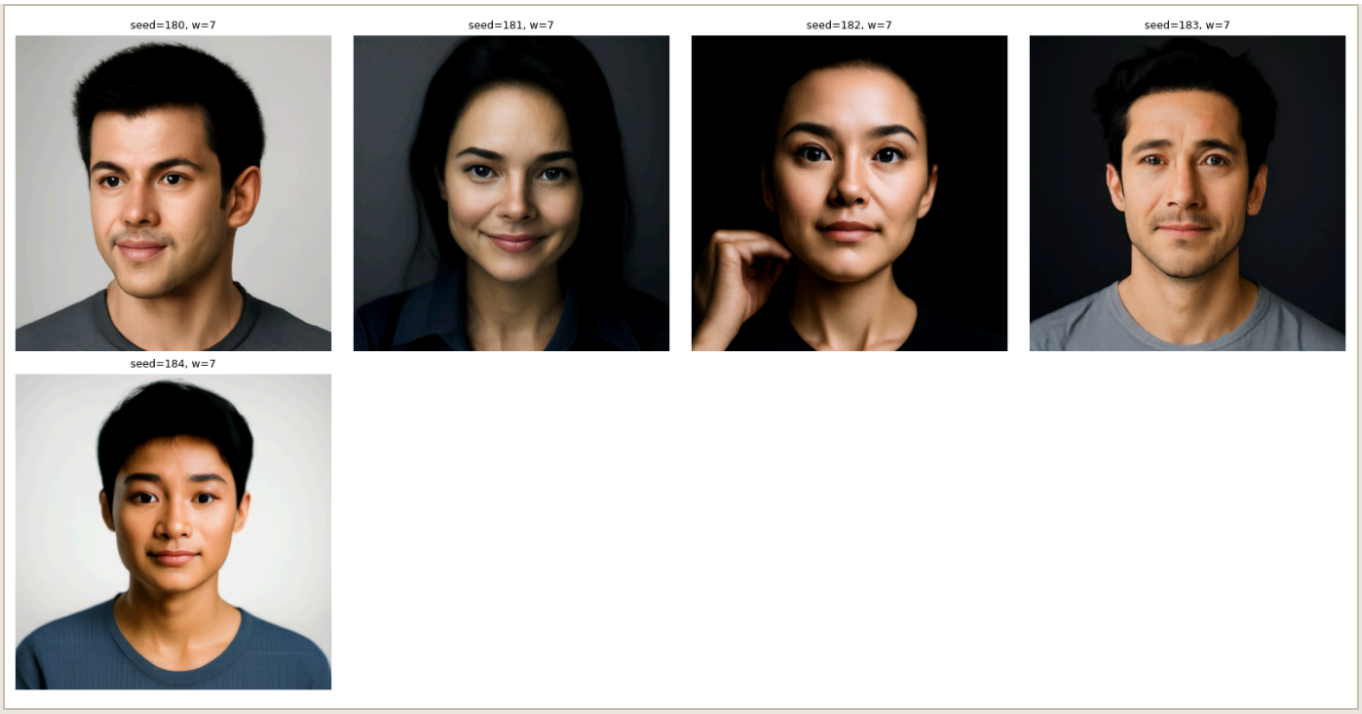
Part 2.1

Classifier-Free Guidance

In this task, I implemented `cfg_velocity`. Classifier-free guidance (CFG) calls the model twice at the same x_t and t, with the prompt (v_c) and with the empty-string null prompt (v_u), and then pushes the velocity further in the direction the prompt adds.

$$v_{\text{cfg}} = v_c + w\,(v_c - v_u)$$

With $w = 0$ this reduces to the plain conditional prediction, and I checked that my implementation returns exactly that. I used $w = 7$ and regenerated the five samples from 1.6 with the *same* starting noises, so the only difference is the guidance.



Same seeds as 1.6 (180–184), now with CFG $w = 7$.

Now all the samples are clean and sharp with more realistic face. Some of the layout carries over from the same noise such as the hand raised to the cheek in seed 182.

Although this approach increases accuracy of the output image, there is a trade-off of image quality and diversity. Guided samples look much more alike (centered face, plain background, similar framing), because CFG pulls each sample toward the most "prompt-like" region of image space.

Part 3.1

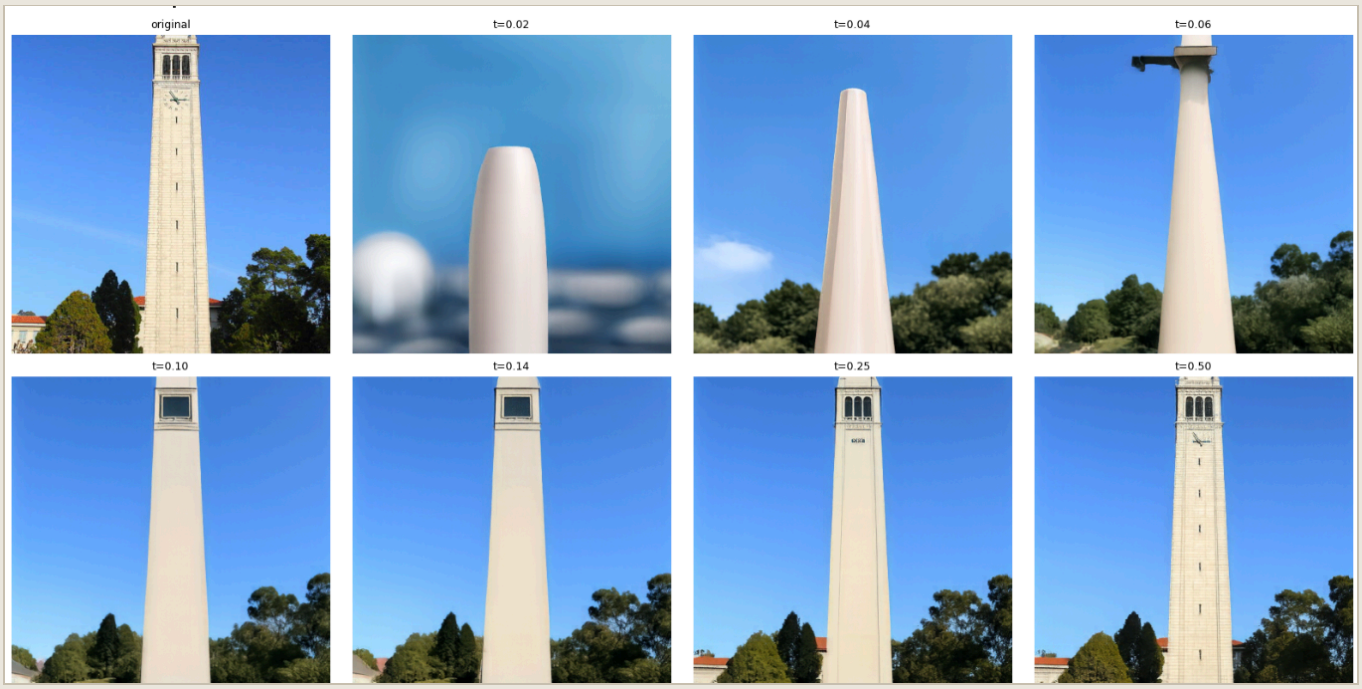
Image-to-Image Editing

To perform image-to-image editing, I add noise to an existing image, and created

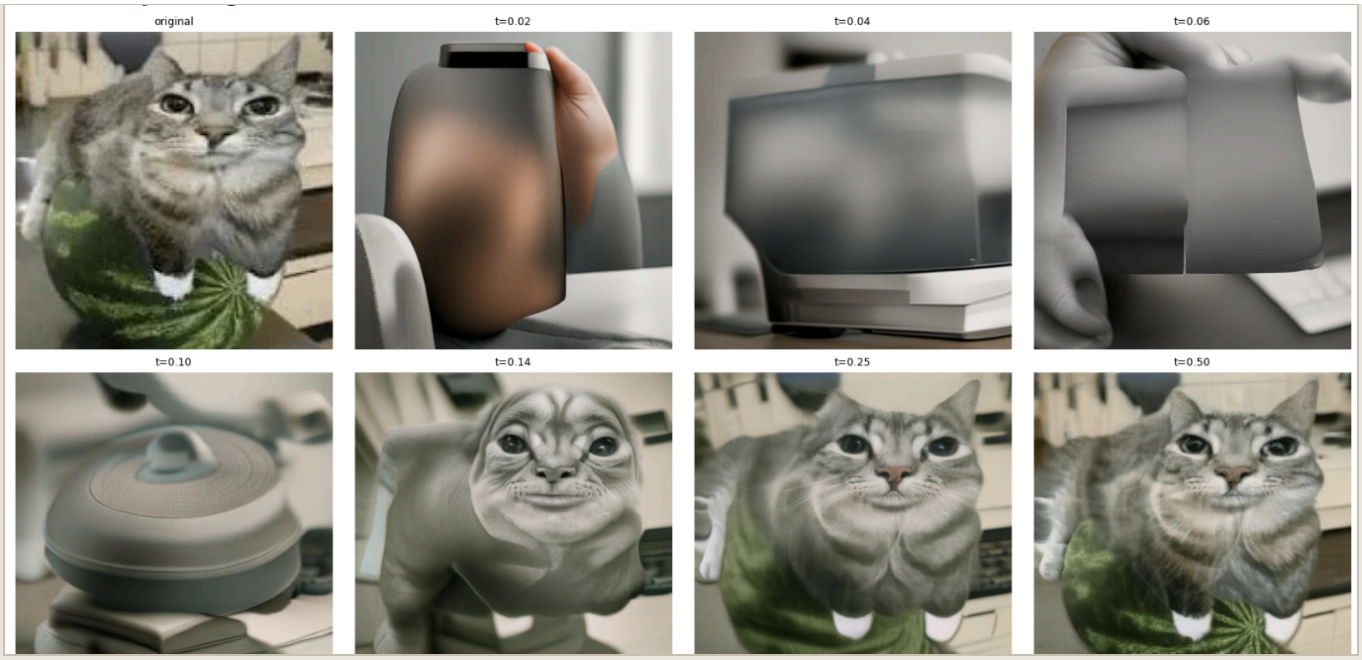
$$x_{t_{\text{start}}} = (1 - t_{\text{start}})\epsilon + t_{\text{start}} x_1$$

Then I run the Euler sampler with CFG from t_{start} to 1. The model has to "fill in" what the noise destroyed, so the result is a new image that keeps some of the original. I used the starting times $0.02, 0.04, 0.06, 0.10, 0.14, 0.25, 0.50$ with the prompt "a high quality photo", $w = 7$ and $h = 0.02$.

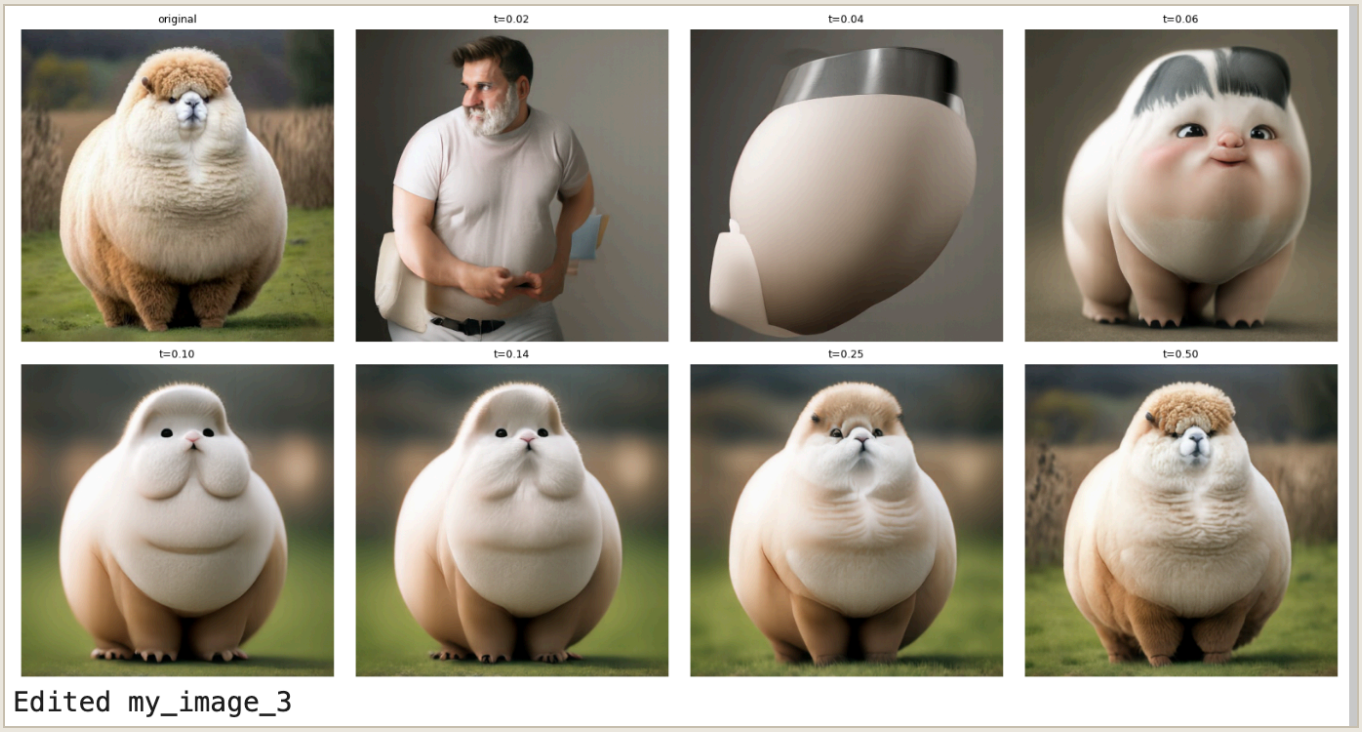
A small t_{start} means almost pure noise, so only the coarsest information (overall brightness and color layout) survives. A large t_{start} keeps most of the original image.



The Campanile. Original, then $t_{\text{start}} = 0.02$ through 0.50.



My cat image. Original, then $t_{\text{start}} = 0.02$ through 0.50.



My llama image. Original, then $t_{\text{start}} = 0.02$ through 0.50.

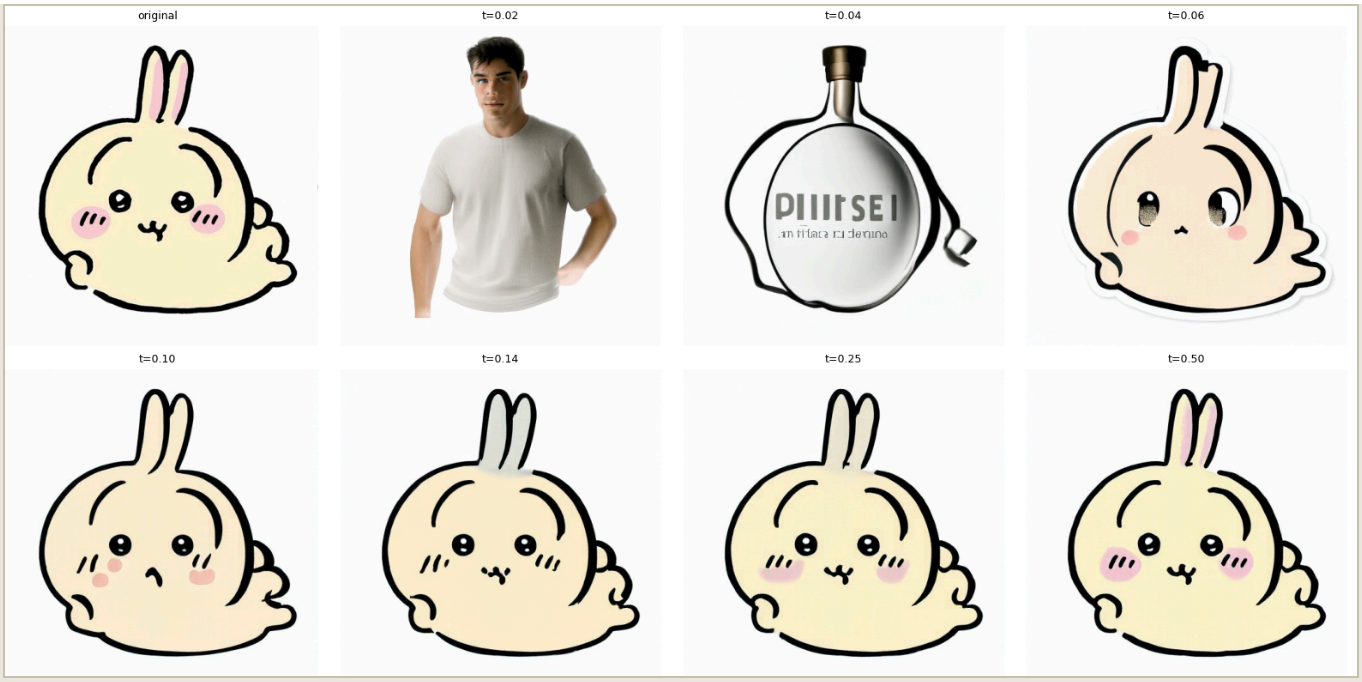
All three show the same progression. At $t_{\text{start}} = 0.02$ to 0.06 the model displays something unrelated that only shares the color, but at around $t_{\text{start}} = 0.10$ to 0.14 the subject's silhouette is restored. By 0.25 and 0.50 the edit is a close, slightly cleaned-up version of the original.

Hand-Drawn and Web Images

I repeated the same procedure on one image from the web (a Chiikawa character) and two of the drawings (Momonga and Usagi), using the same seven starting times.

Web image. Original, then $t_{\text{start}} = 0.02$ through 0.50.

A drawing of Momonga. Original, then $t_{\text{start}} = 0.02$ through 0.50.



At the smallest starting times the model displays something that are very random, such as tshirt and perfume bottle. At $t_{\text{start}} = 0.04$ to 0.06 it recovers outline with real-looking objects, such as a rubber band or a pair of glasses with button eyes. From about 0.10 on, the model keeps the line-art style and the characters come back. It only loses small details, like the blush marks and the exact facial expression, until those return around 0.25 to 0.50.

Part 3.2

Text-Guided Image Editing

In this task, I used the same `edit` function as 3.1, but replaces the generic photo prompt with a prompt that describes the edit I want. The noise erases part of the image and the new prompt decides what fills it back in. I used the same seven starting times on the Campanile, the Chiikawa image and Usagi images.

Image	Prompt
Campanile	“a beautiful watercolor painting of the UC Berkeley Campanile surrounded by cherry blossoms”
Chiikawa	“a cute fluffy cat sitting in a magical forest, whimsical storybook illustration”
Usagi	“a cute llama wearing a colorful knitted sweater, in a sunny meadow”

The Campanile with “a beautiful watercolor painting of the UC Berkeley Campanile surrounded by cherry blossoms”. Original, then $t_{\text{start}} = 0.02$ through 0.50.

Chiikawa with “a cute fluffy cat sitting in a magical forest, whimsical storybook illustration”. Original, then $t_{\text{start}} = 0.02$ through 0.50.

My drawing of Usagi with “a cute llama wearing a colorful knitted sweater, in a sunny meadow”. Original, then $t_{\text{start}} = 0.02$ through 0.50.

At low t_{start}, the model generates image that is closely matches with the prompt ($0.02, 0.04$). At 0.25 and 0.50 the original wins and only the texture changes, such as

a light watercolor wash on the tower. The best edits are in between. Around 0.06 to 0.10 Usagi keeps its ears and round body but gains a knitted hat and llama fur, and Chiikawa keeps its pose while picking up whiskers and a forest background. The Campanile looks best around 0.10 to 0.14, where it keeps its real shape but is painted in watercolor.

Part 3.3

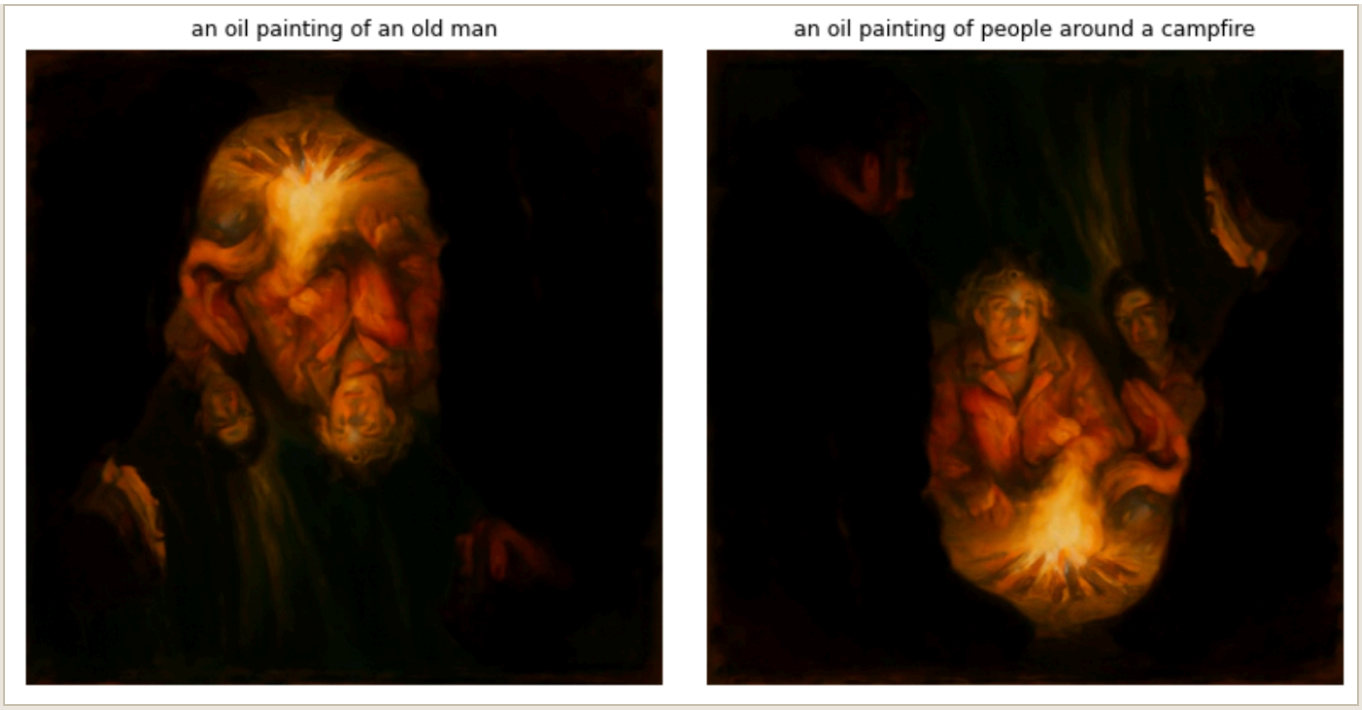
Visual Anagrams

A visual anagram is a single image that shows one thing upright and something else when rotated by 180°. Let F be a 180° rotation, which flips both spatial axes. At every Euler step I compute two guided velocities and average them:

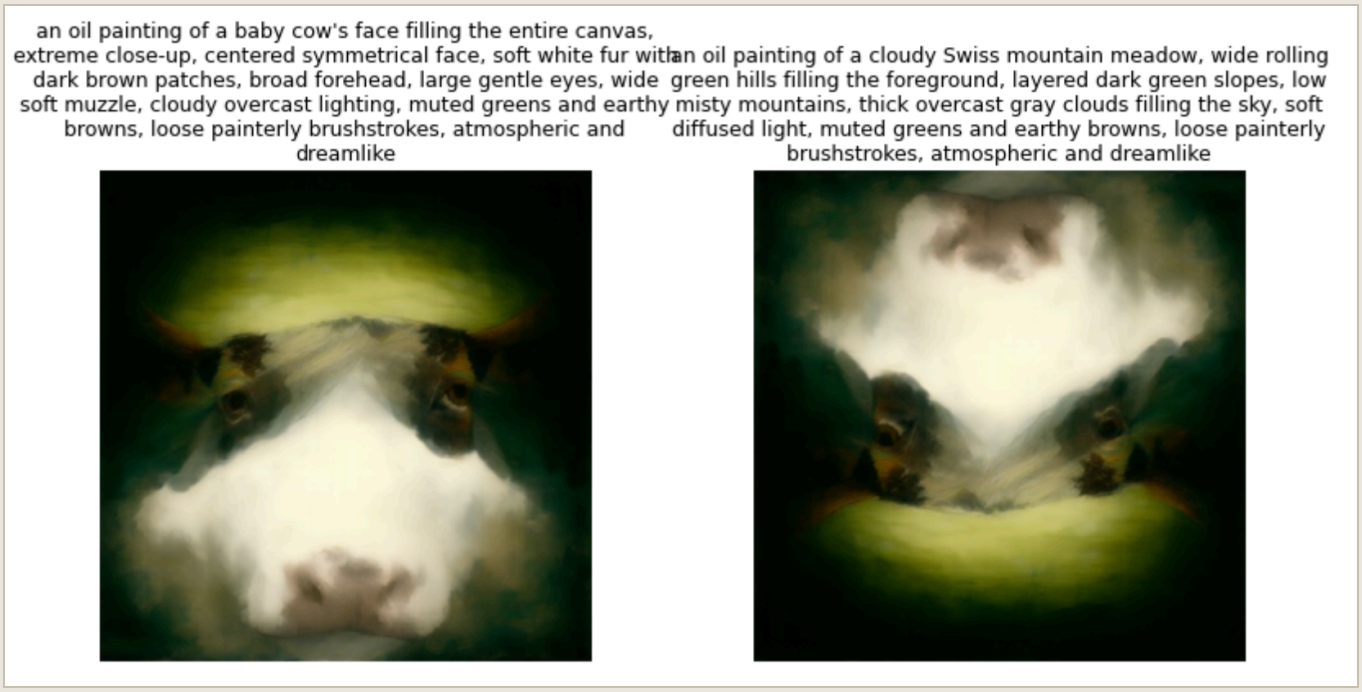
$$v = \tfrac{1}{2}\Big(v_{\text{cfg}}(x_t, t, c_a) + F\big(v_{\text{cfg}}(F(x_t), t, c_b)\big)\Big)$$

I implemented this in `visual_anagrams`. The first term denoises the image toward prompt a as it is. The second rotates the image, denoises it toward prompt b, and rotates that velocity back. Since $F(F(x)) = x$ (I checked this), both views are pushed toward their own prompt at every step. I integrate from pure noise with the same Euler sampler.

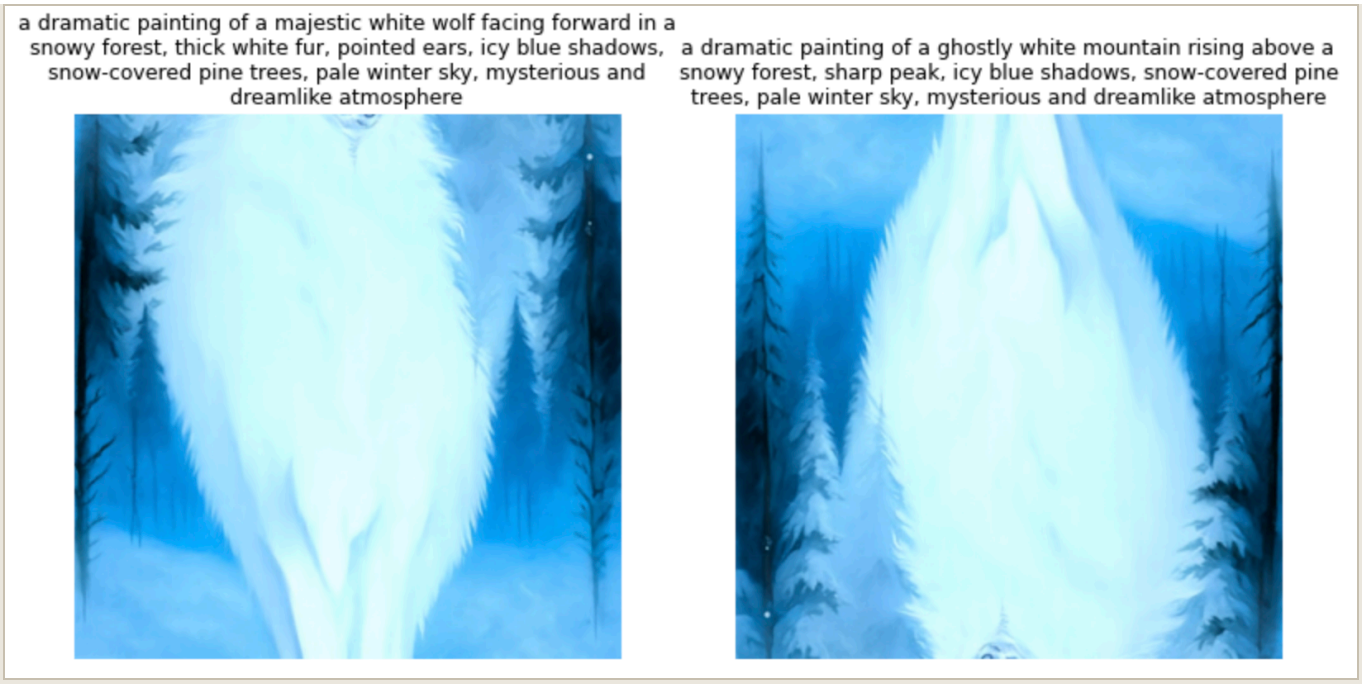
Prompt pairs: “an oil painting of an old man” / “an oil painting of people around a campfire”, a baby cow's face / a cloudy Swiss mountain meadow, and a white wolf / a white mountain in a snowy forest (full prompts in the figures).



Upright: “an oil painting of an old man”. Rotated 180°: “an oil painting of people around a campfire”.



Upright: an oil painting of a baby cow's face. Rotated 180°: an oil painting of a cloudy Swiss mountain meadow.



Upright: a dramatic painting of a white wolf in a snowy forest. Rotated 180°: a dramatic painting of a white mountain above a snowy forest.

The model doesn't closely follow the details in my prompt sentences, so getting both views to work took several tries (7 attempts for the cow / meadow pair and 5 for the wolf / mountain pair)

Part 3.4

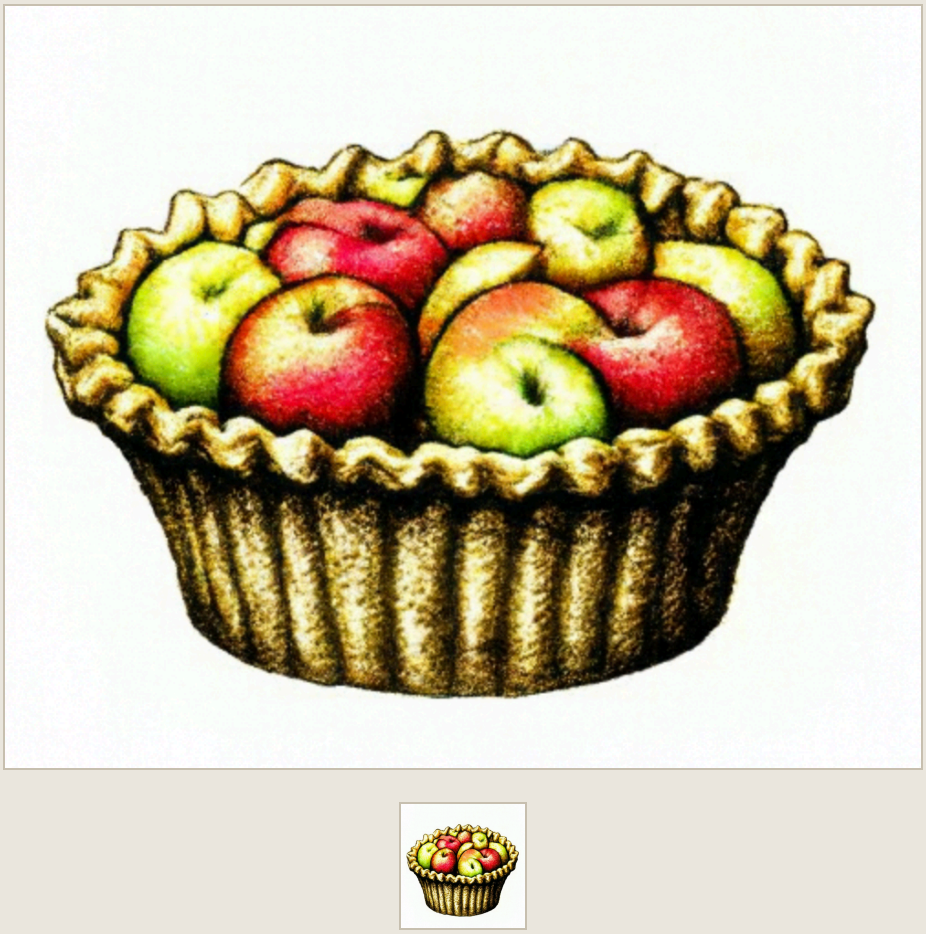
Hybrid Images

This is the generative version of the hybrid images from Project 2, adapted from Factorized Diffusion and implemented in `make_hybrids`. At each Euler step I compute guided velocities for both prompts at the same x_t and t. I keep the low frequencies of prompt a and the high frequencies of prompt b:

$$v = \mathrm{LP}(v_a) + \big(v_b - \mathrm{LP}(v_b)\big)$$

LP is a Gaussian blur with $\sigma = 16$ and kernel size 265 at 512×512. Seen from far away (or at a small size) only the low frequencies are visible, so prompt a shows. Up close, the high-frequency detail of prompt b takes over.

Prompt pairs (distant / close): “a lithograph of basketfull of apples” / “a lithograph of an apple pie”, and “a lithograph of butterflies” / “a lithograph of a flower”.



Top: full size, close prompt “a lithograph of an apple pie”. Bottom: reduced to 64×64, distant prompt “a lithograph of basketfull of apples”.

Top: full size, close prompt “a lithograph of a flower”. Bottom: reduced to 64×64, distant prompt “a lithograph of butterflies”.

Project 3 · Part B

Flow Matching from Scratch

Training a one-step denoiser and then time- and class-conditioned flow matching UNets on MNIST.

Part B · 1.1

Implementing the UNet

I built the UNet from the standard operations in the spec:

Block	Implementation
Conv	`Conv2d(3, stride 1, pad 1)` → BatchNorm → GELU. Keeps the resolution.
DownConv	`Conv2d(3, stride 2, pad 1)` → BatchNorm → GELU. Halves the resolution.
UpConv	`ConvTranspose2d(4, stride 2, pad 1)` → BatchNorm → GELU. Doubles the resolution.
ConvBlock	Two Convs in a row.
DownBlock / UpBlock	DownConv or UpConv followed by a ConvBlock.
Flatten	`AvgPool2d(7)` → GELU. Turns 7×7 into 1×1.
Unflatten	`ConvTranspose2d(7, stride 7, pad 0)` → BatchNorm → GELU. Turns 1×1 back into 7×7.

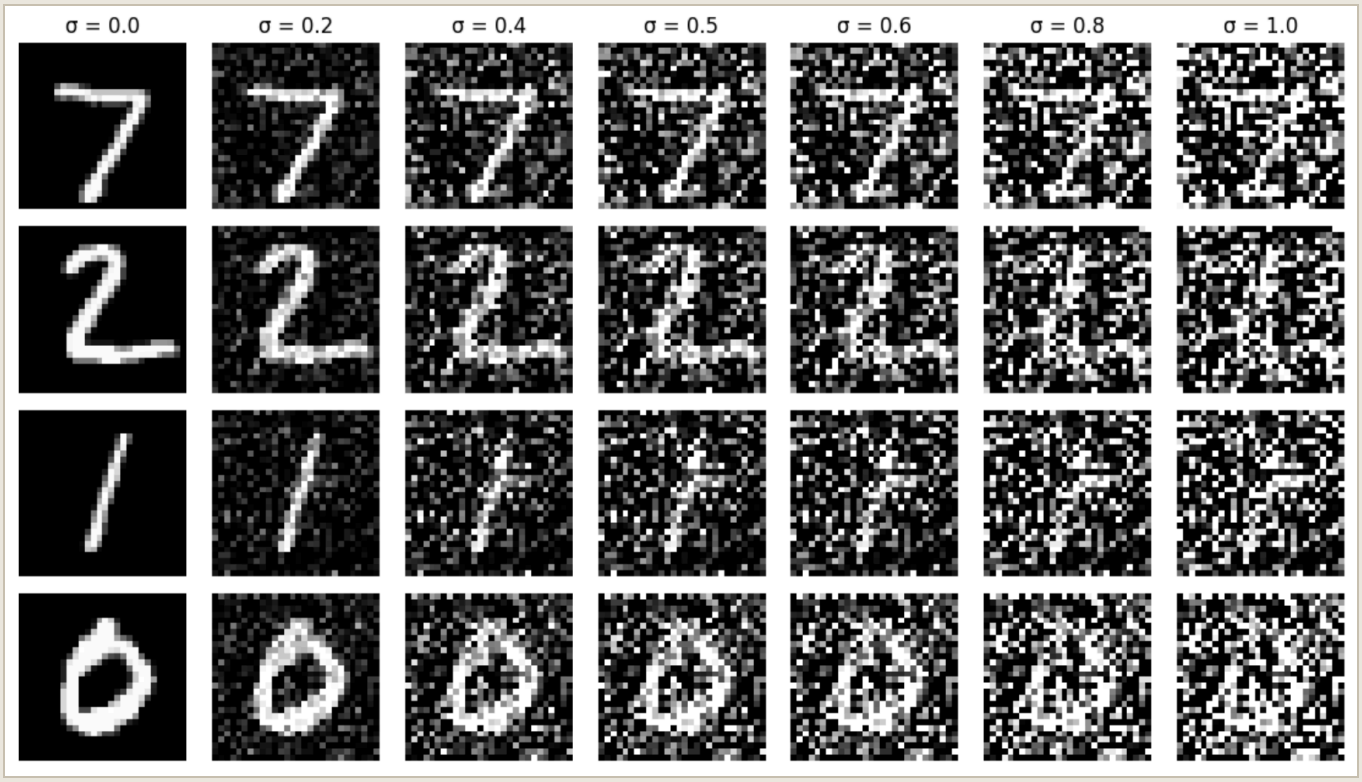
The unconditional UNet encodes a 28×28 digit with a ConvBlock (D channels), then two DownBlocks that go to 14×14 (D channels) and 7×7 ($2D$ channels). Flatten averages the 7×7 map down to a single $2D$-channel vector, and Unflatten expands it back to 7×7. This bottleneck lets every output pixel see the whole digit. The decoder concatenates the unflattened map with the matching encoder map (the

skip connection) and upsamples with an UpBlock to 14×14. It concatenates again and upsamples to 28×28, then concatenates with the first encoder map and runs a final ConvBlock. A 3×3 convolution maps D channels back to one output channel. I checked that a batch of shape $(8, 1, 28, 28)$ comes out with the same shape.

Part B · 1.2

Using the UNet to Train a Denoiser

The denoiser D_θ is trained to map a noisy digit $z = x + \sigma\epsilon$, with $\epsilon \sim \mathcal{N}(0, I)$, back to the clean digit x by minimizing the L2 loss $\|D_\theta(z) - x\|^2$. MNIST pixels are in $[0, 1]$. Below, four test digits are noised with one fixed noise sample ϵ scaled by increasing σ.

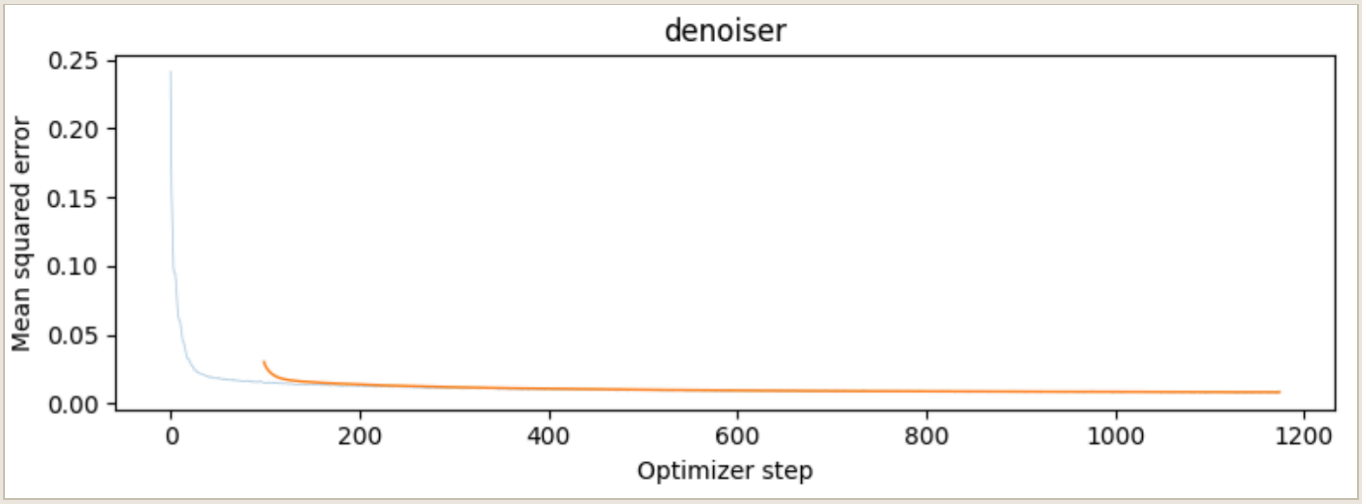


The noising process $z = x + \sigma\epsilon$ for $\sigma = 0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0$.

At $\sigma = 0.2$ the digits are still easy to read. By $\sigma = 0.5$ the strokes are broken up by speckle but still recognizable. At $\sigma = 1.0$ the digit is barely visible in the noise.

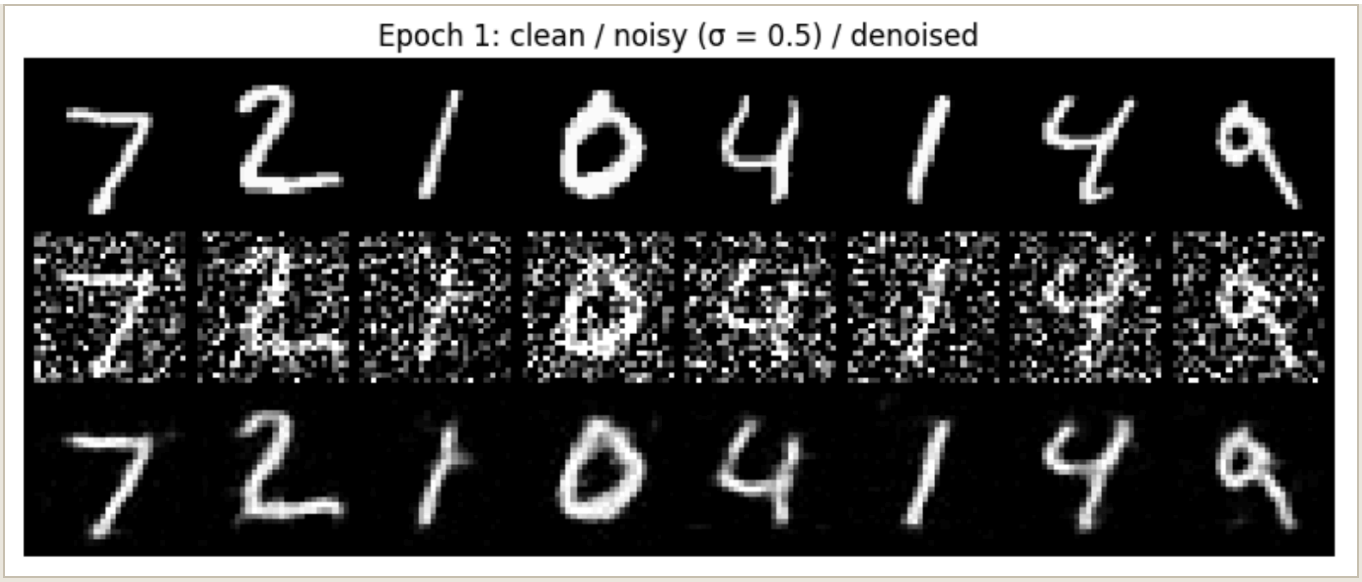
1.2.1 Training

I trained the UNet with $D = 128$ on the MNIST training set for 5 epochs, with batch size 256 and Adam at a learning rate of 10^{-4}. The dataloader shuffles the data, and I noise each batch with $\sigma = 0.5$ only when it is fetched. That way the network sees fresh noise on every pass instead of a fixed noisy dataset.

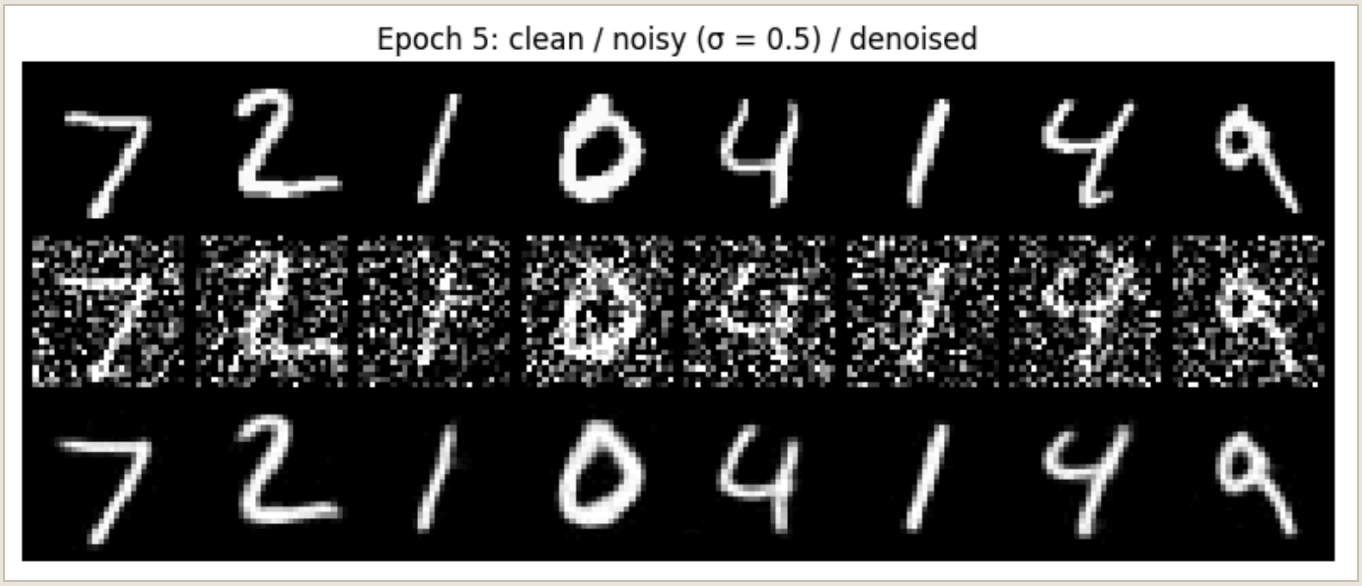


Training loss for every iteration (235 per epoch, 1,175 total). Blue: per-iteration loss. Orange: 100-step moving average.

The loss falls from about 0.24 to around 0.02 within the first ~50 iterations, then keeps slowly decreasing to about 0.01 by the end of epoch 5.



After epoch 1. Rows: clean test digits, noisy input ($\sigma = 0.5$), denoised output.



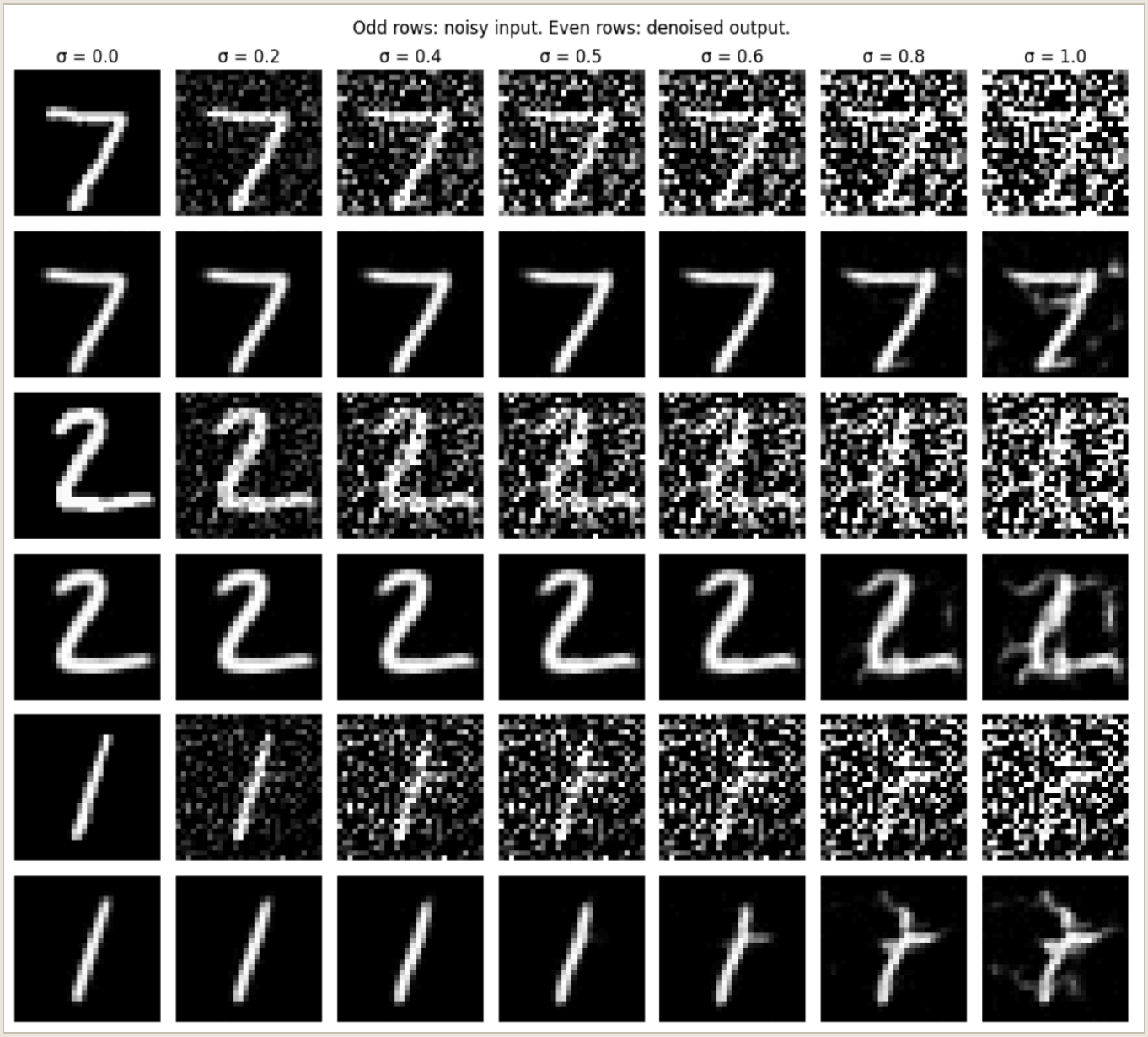
After epoch 5, with the same digits and the same noise.

After one epoch every digit is already recognizable, but the outputs are a little blurry and have small artifacts, such as an extra tick on the first 1 and faint smudges in the background. After five epochs the strokes are clean, sharp and close to the originals.

1.2.2 Out-of-Distribution Testing

The denoiser only saw $\sigma = 0.5$ during training. To see how it handles other noise levels, I took three test digits and noised each one with the same noise pattern scaled

to every σ from 0.0 to 1.0.

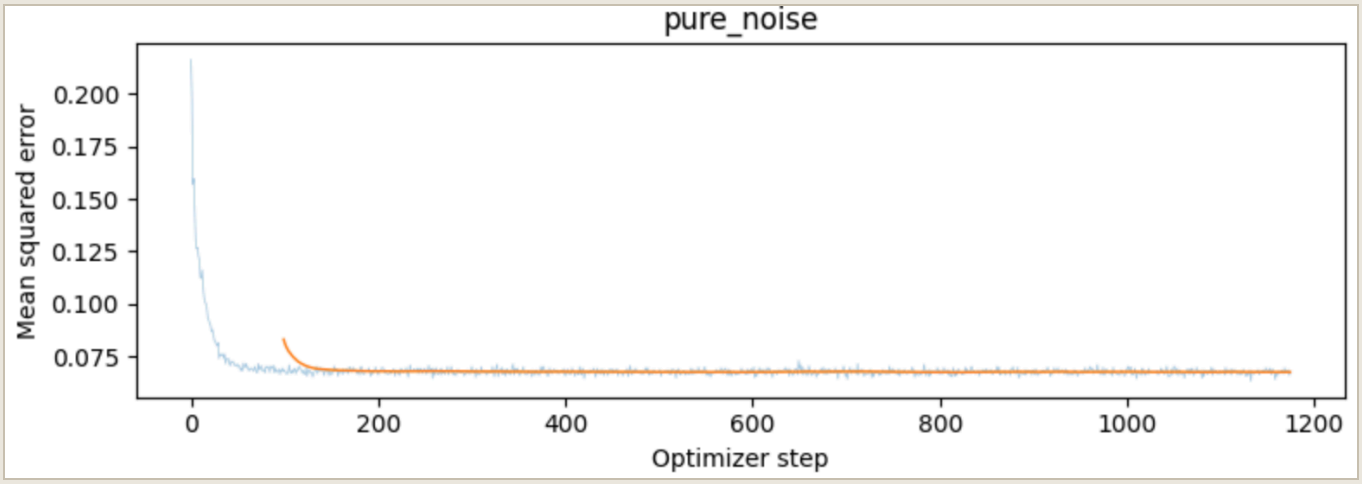


For each digit, the top row is the noisy input and the bottom row is the denoiser output, for $\sigma = 0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0$.

For $\sigma \leq 0.5$ (at or below the training level) the outputs are clean and nearly identical to the original digits. At $\sigma = 0.6$ the 1 already picks up a small extra stroke. At $\sigma = 0.8$ and 1.0, beyond what the model was trained on, it starts inventing structure: the 1 turns into something like a 7 or a 4, the 2 gets extra loops, and the 7 picks up a stray crossbar. With more noise than it has ever seen, the network mistakes some noise for strokes and fills them in as parts of other digits.

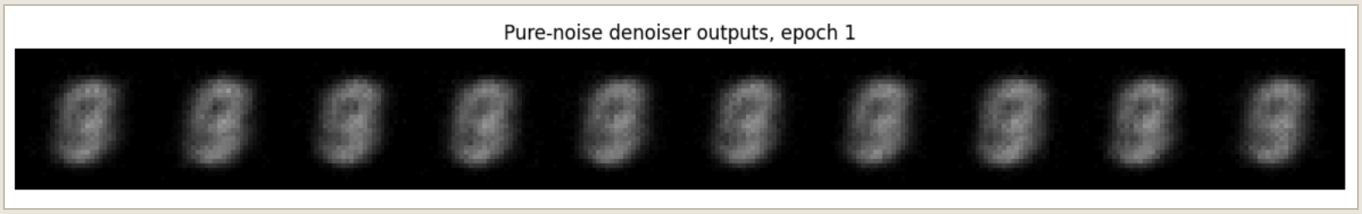
1.2.3 Denoising Pure Noise

To turn the denoiser into a generator, I trained a fresh $D = 128$ UNet with the same settings (5 epochs, batch 256, Adam 10^{-4}). This time the input is pure Gaussian noise $z = \epsilon$, independent of the target digit.

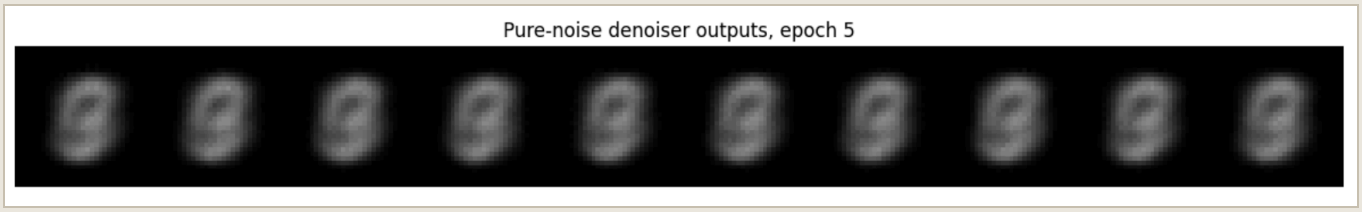


Training loss for every iteration when the input is pure noise. Orange: 100-step moving average.

Unlike in 1.2.1, the loss levels off at about 0.068 after roughly 100 iterations and doesn't improve after that. The input carries no information about which digit is the target, so there is nothing more to learn.



Outputs for 10 different pure-noise inputs, after epoch 1.



The same 10 inputs after epoch 5.

For comparison: the pixel-wise mean of all 60,000 training images.

Every output is the same gray, blurry blob, no matter what noise goes in. It is an oval of light in the middle with a darker hole near the top. It doesn't look like any one digit, more like a 0, 3, 8 and 9 drawn on top of each other. The epoch-1 outputs are a little grainy and differ slightly from each other. By epoch 5 they are smooth and essentially identical, and they look almost exactly like the mean training image.

This is what MSE loss predicts. For a given input, the prediction that minimizes squared error is the average of all targets that could go with it. Since the noise is independent of the digit, every training image is an equally likely target for every input, so the best the model can do is output one fixed image: the pixel-wise mean, or centroid, of the training set. That is why the outputs have no relationship to the input and look like an average of all digits 0–9. It is also why the loss levels off. With a constant prediction, the remaining error is just how much the digits vary around their mean, and training can't reduce that. One-step denoising of pure noise therefore can't generate new digits, which motivates the iterative flow matching in Part 2.

Part B · 2.1

Adding Time Conditioning to the UNet

One-step denoising can't generate, so I switched to flow matching. A training point is a linear interpolation between noise $x_0 \sim \mathcal{N}(0, I)$ and a clean digit x_1:

$$x_t = (1 - t)\, x_0 + t\, x_1, \qquad u(x_t, t) = \frac{dx_t}{dt} = x_1 - x_0$$

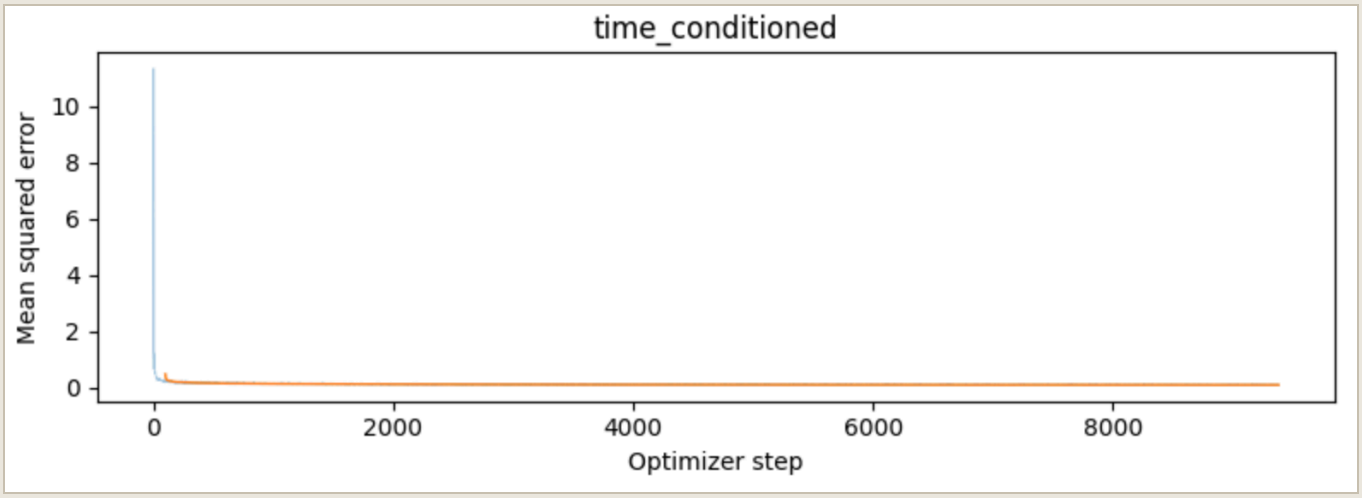
The UNet $u_\theta(x_t, t)$ learns to predict this flow with the loss $\|u_\theta(x_t, t) - (x_1 - x_0)\|^2$.

To give the network t, I added the FCBlock (`Linear` → GELU → `Linear`) to the same UNet as in 1.1. Two FCBlocks embed the scalar $t \in [0, 1]$: `fc1_t` outputs $2D$ values and `fc2_t` outputs D values. Each is broadcast over the spatial dimensions and *multiplies* a decoder feature map, following the spec: `unflatten = unflatten * t1` at the bottleneck and `up1 = up1 * t2` after the first UpBlock. This lets the network scale its features differently depending on how noisy the input is.

Part B · 2.2

Training the Time-Conditioned UNet

For each batch I sample fresh noise x_0 and a time $t \sim U[0, 1]$ for each image, build x_t, and regress the UNet's output onto $x_1 - x_0$. I used the recommended settings: $D = 64$, batch size 64, 10 epochs, Adam with initial learning rate 10^{-2}, and `ExponentialLR` with $\gamma = 0.1^{1/10}$, stepped once per epoch. I saved checkpoints after epochs 1, 5 and 10 for sampling.

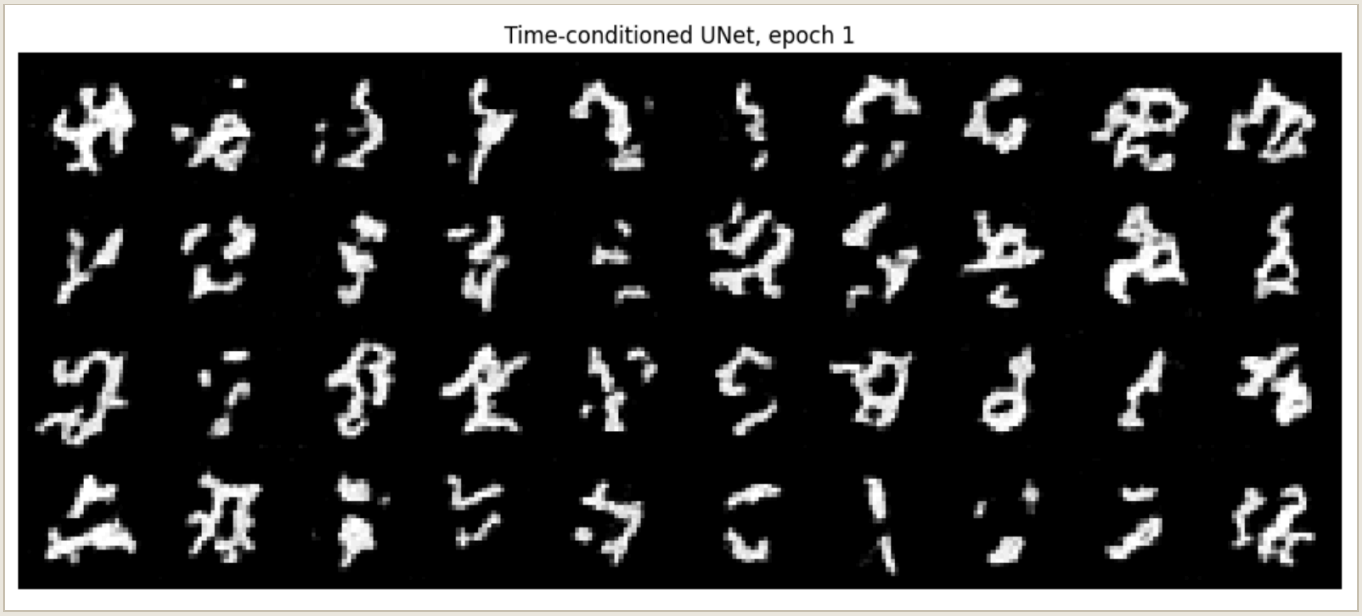


Flow-matching loss for every iteration of the time-conditioned UNet (938 per epoch, 10 epochs). Orange: 100-step moving average.

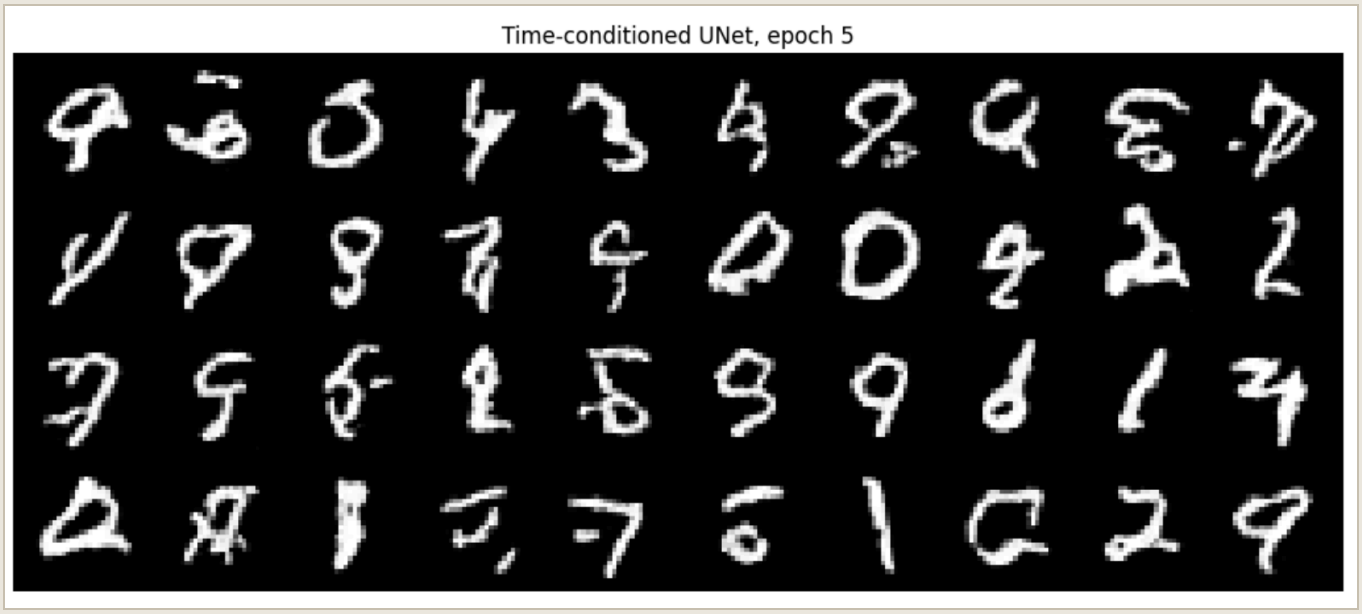
The very first iterations have a loss above 10. This is most likely due to the large initial learning rate of 10^{-2} combined with the multiplicative time modulation, which can make the first updates overshoot. The loss collapses within the first ~100 steps and then decreases slowly for the rest of training as the learning rate decays. It stays well above the denoiser's loss from 1.2.1, because the target $x_1 - x_0$ includes the noise itself, which is hard to predict when t is close to 1.

Sampling from the Time-Conditioned UNet

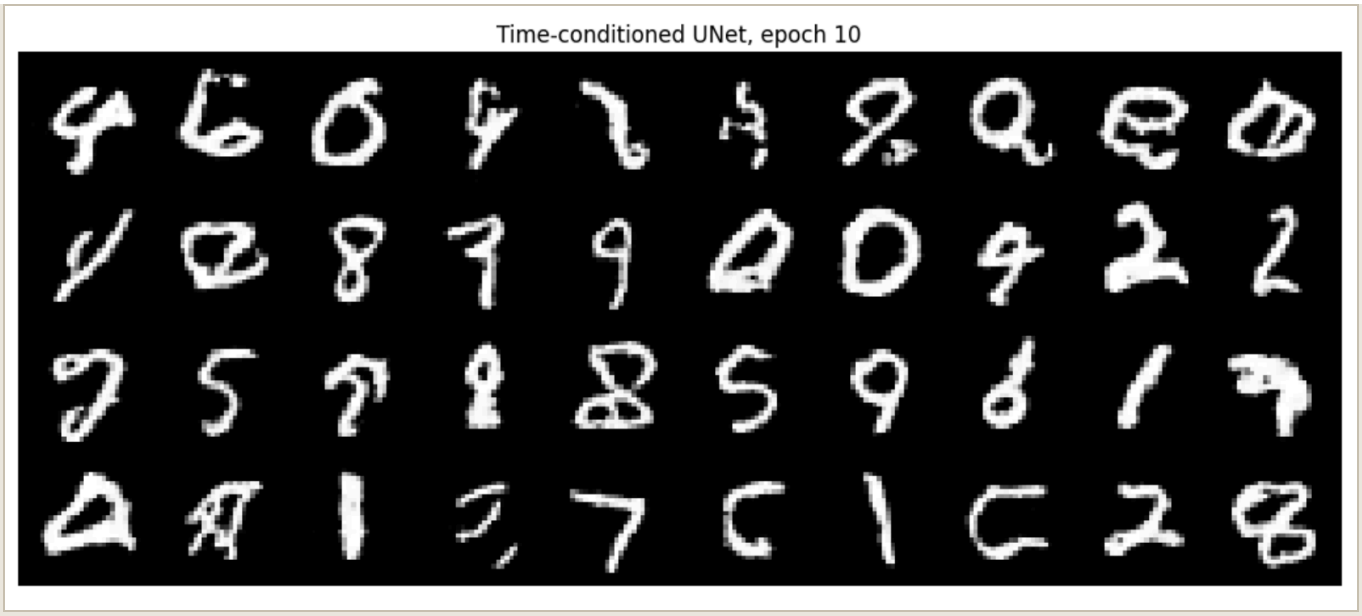
To sample, I start from pure noise at $t = 0$ and take 50 Euler steps of size $1/50$, $x \leftarrow x + \frac{1}{50}\, u_\theta(x, t)$, ending at $t = 1$.



40 samples after epoch 1.



After epoch 5, with the same 40 starting noises.



After epoch 10, with the same 40 starting noises.

After one epoch the samples are white blobs and squiggles on a clean black background. They have the stroke thickness of MNIST digits but don't form any digit. By epoch 5 roughly half of them are readable (several 0s, 2s, 9s, a 1, a 3), and the rest are tangled or mix two digits together. After epoch 10 most samples are legible digits (0, 1, 2, 5, 7, 8, 9 all appear) with smoother, more confident strokes. A few are still malformed, such as unfinished loops or a digit that looks half 4 and half 9.

The same starting noise often turns into the same digit at epochs 5 and 10, just cleaner. For example, the 0 and the two 2s in the second row, and the 1 in the bottom row. This shows that the overall shape is decided by the noise early in sampling, and further training mostly sharpens it. Without a class label, the model sometimes can't decide which digit a noise sample should become, which is where the mixed-up samples come from.

Part B · 2.4

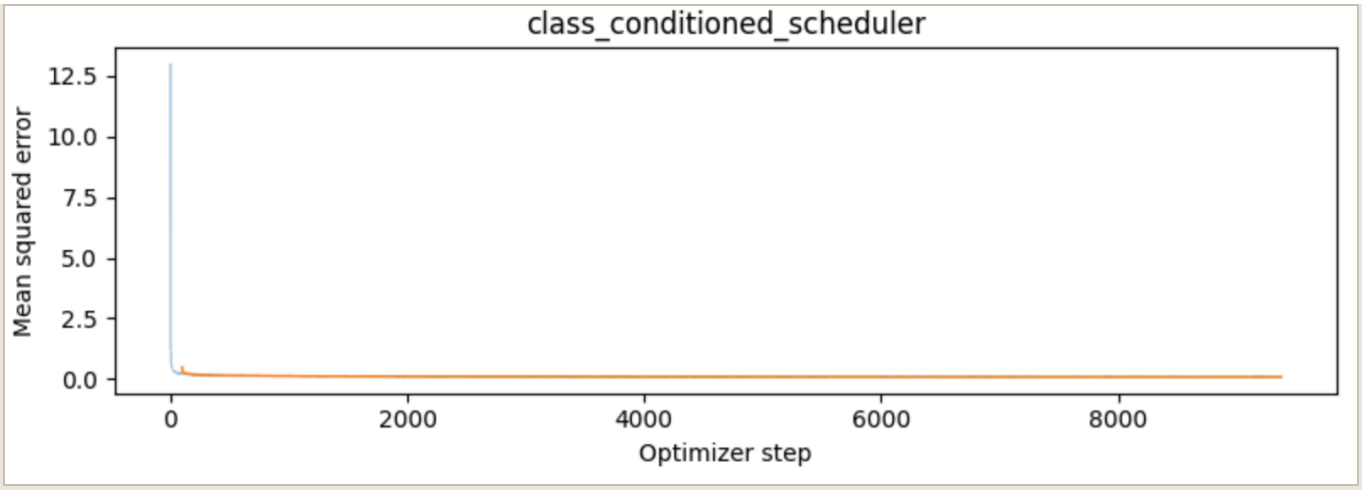
Adding Class Conditioning to the UNet

To control which digit gets generated, I also conditioned the UNet on the class $c \in \{0, \ldots, 9\}$. The class is a one-hot vector of length 10, embedded by two more FCBlocks (`fc1_c` with $2D$ outputs, `fc2_c` with D outputs). Class and time are combined at the same two places as before: `unflatten = c1 * unflatten + t1` and `up1 = c2 * up1 + t2`. During training, the one-hot vector is set to all zeros with probability $p_{\text{uncond}} = 0.1$, so the same network also learns the unconditional flow needed for classifier-free guidance. At sampling time the unconditional prediction uses that same all-zero vector.

Part B · 2.5

Training the Class-Conditioned UNet

Training is the same as for the time-only model, except the class labels are passed in and randomly dropped. I followed the recommended settings: $D = 64$, batch size 64, 10 epochs, Adam with initial learning rate 10^{-2}, and `ExponentialLR` with $\gamma = 0.1^{1/10}$, so the learning rate falls by 10× over the 10 epochs. I called `scheduler.step()` once per epoch.

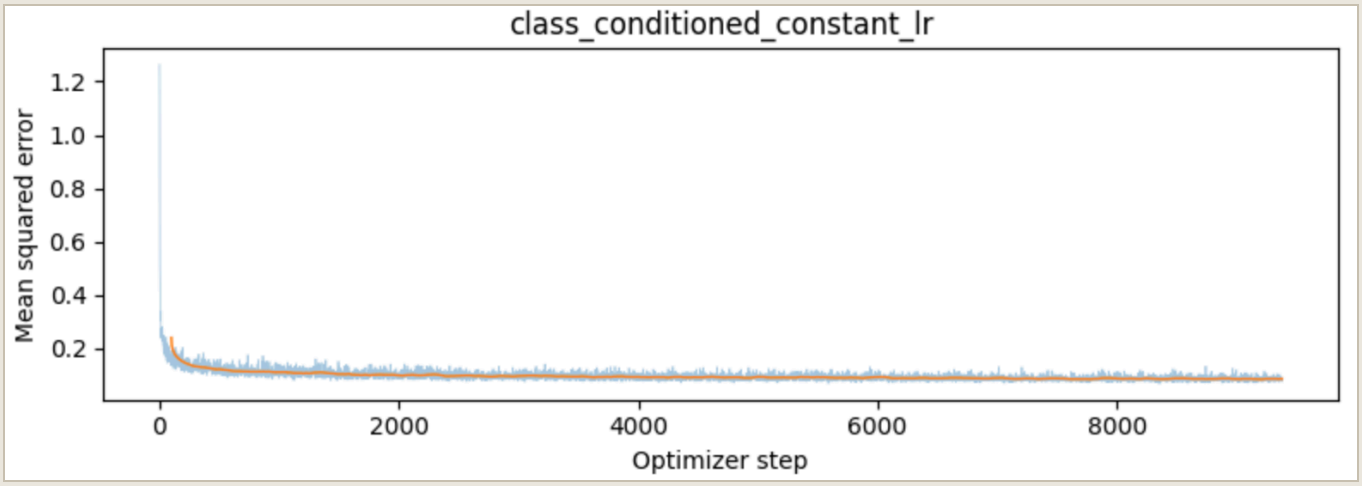


Flow-matching loss for every iteration of the class-conditioned UNet with the exponential scheduler (938 per epoch, 10 epochs). Orange: 100-step moving average.

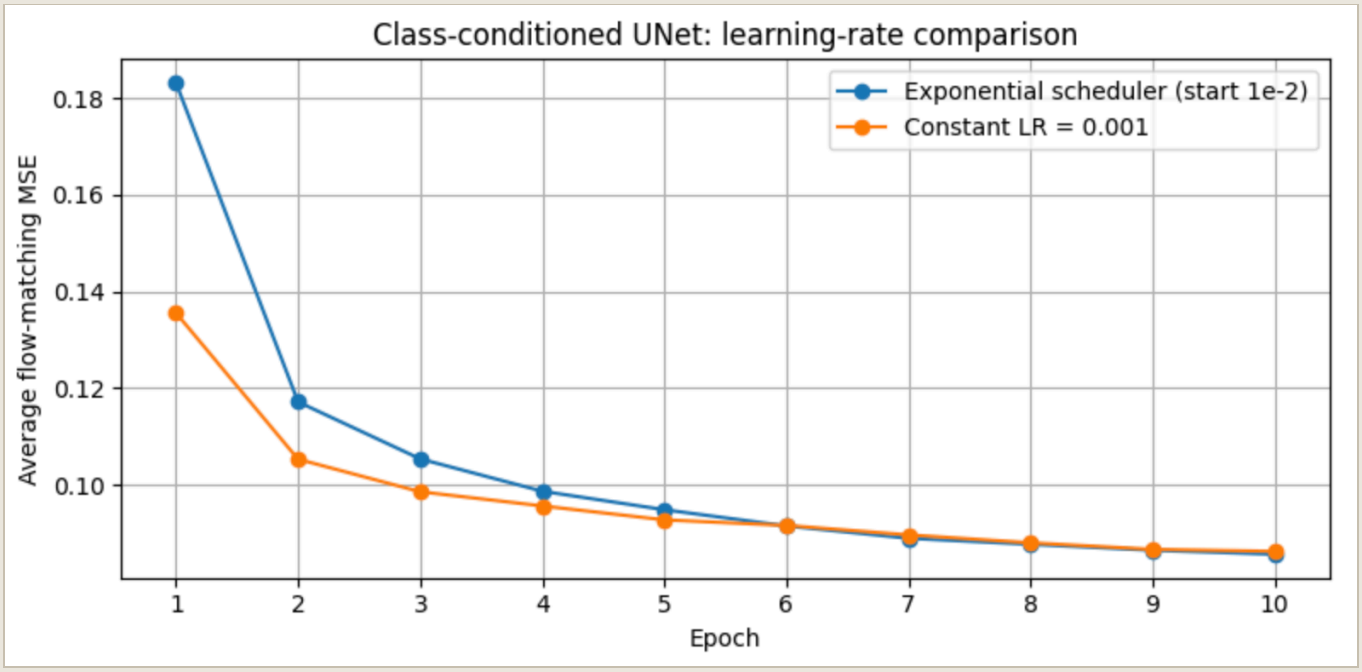
As with the time-only model, the first few iterations have a very large loss (above 12) at the 10^{-2} learning rate. The loss collapses within about 100 steps and then decreases slowly for the rest of training.

Removing the Learning Rate Scheduler

To get rid of the scheduler, I trained a second, independent model with exactly the same settings but a **constant learning rate of** 10^{-3}, 10× lower than the scheduled model's starting rate. Without a schedule, 10^{-2} would stay too large late in training. A lower rate makes the early updates gentler, and it is already close to where the schedule ends after 10 epochs (about 1.3×10^{-3}), so the model can settle into a minimum without any decay.



Flow-matching loss for every iteration with a constant learning rate of 10^{-3} and no scheduler.



Average loss per epoch for both models. Blue: exponential scheduler starting at 10^{-2}. Orange: constant 10^{-3}.

With the constant rate, the first loss spike is only about 1.2 instead of above 12, and the loss is lower for the first five epochs (about 0.136 vs. 0.183 after epoch 1). The two curves meet at epoch 6 and stay nearly on top of each other until the end, where both reach about 0.086. A single well-chosen constant learning rate matches the schedule's final loss, and the samples in 2.6 look just as good.

Part B · 2.6

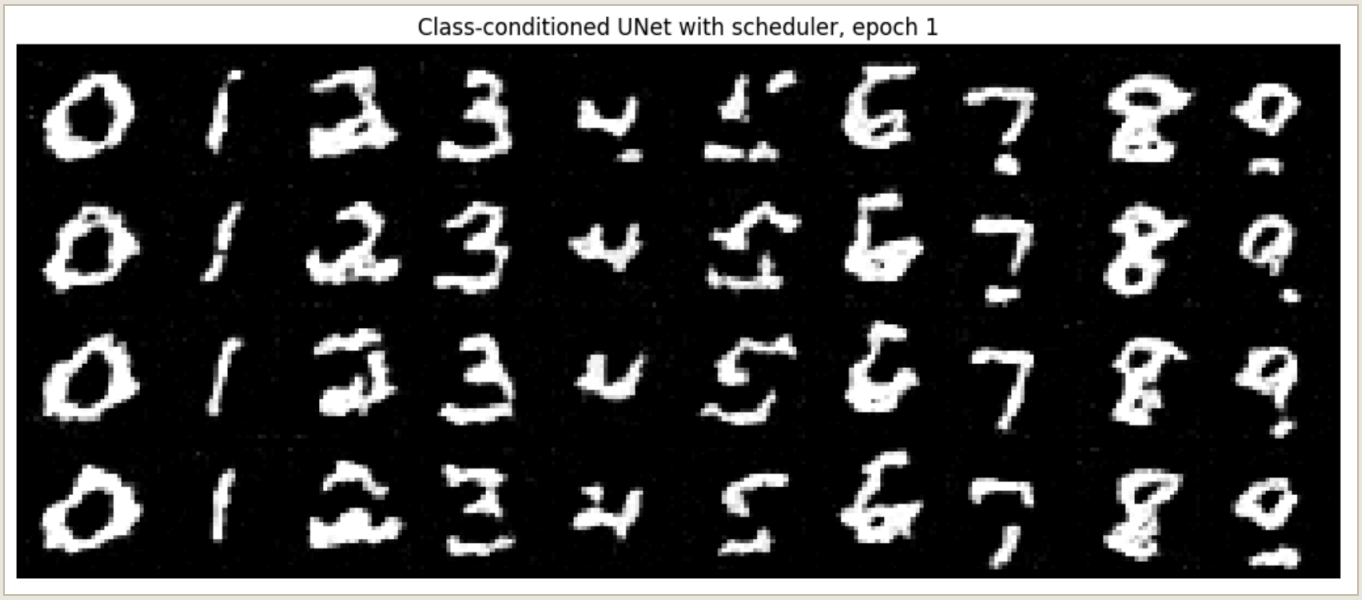
Sampling from the Class-Conditioned UNet

Sampling uses 300 Euler steps from pure noise with classifier-free guidance at scale $\gamma = 5$. At every step I evaluate the velocity with the one-hot class vector c and with

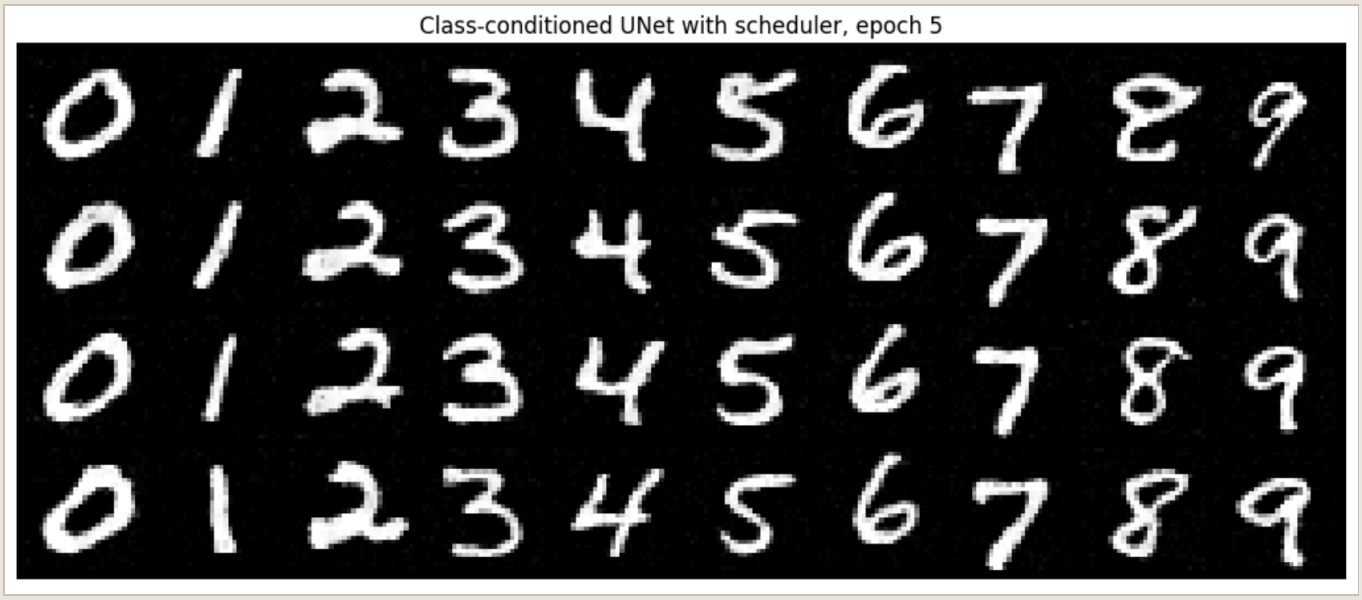
the all-zero vector $\varnothing$, and combine them:

$$u = u_\theta(x_t, t, \varnothing) + \gamma \left(u_\theta(x_t, t, c) - u_\theta(x_t, t, \varnothing) \right)$$

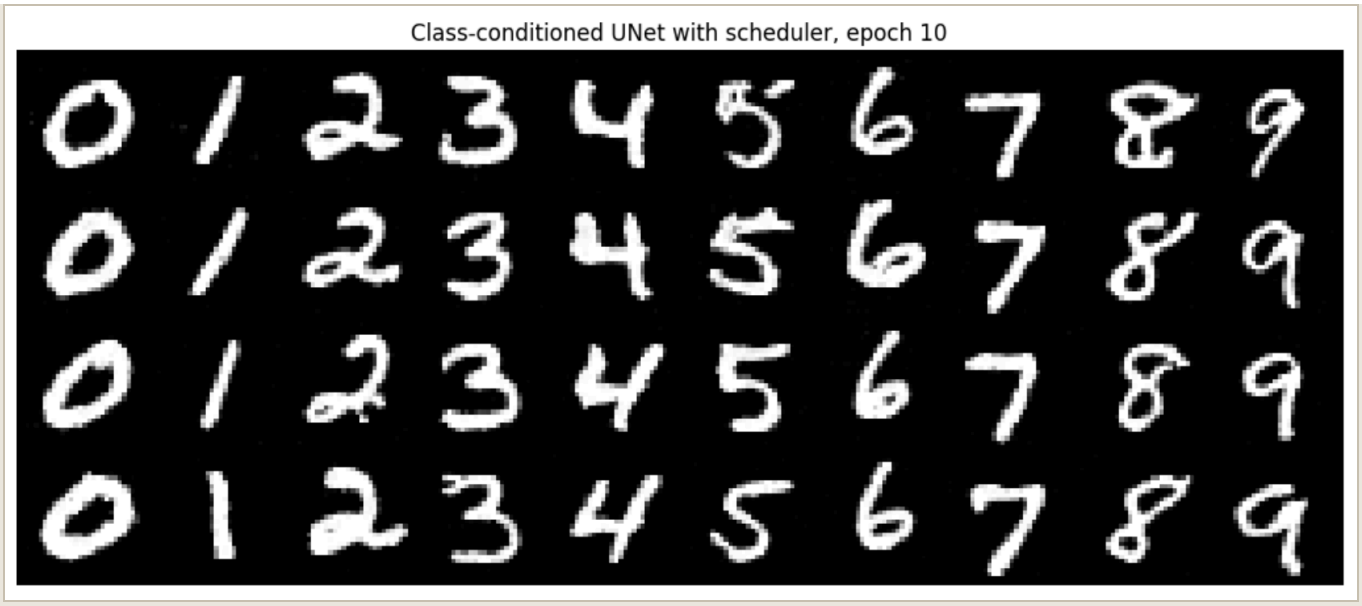
Each grid has one column per digit 0–9 and four samples of each. The starting noise is the same at every checkpoint, so differences between grids come only from training.



With scheduler, after epoch 1.

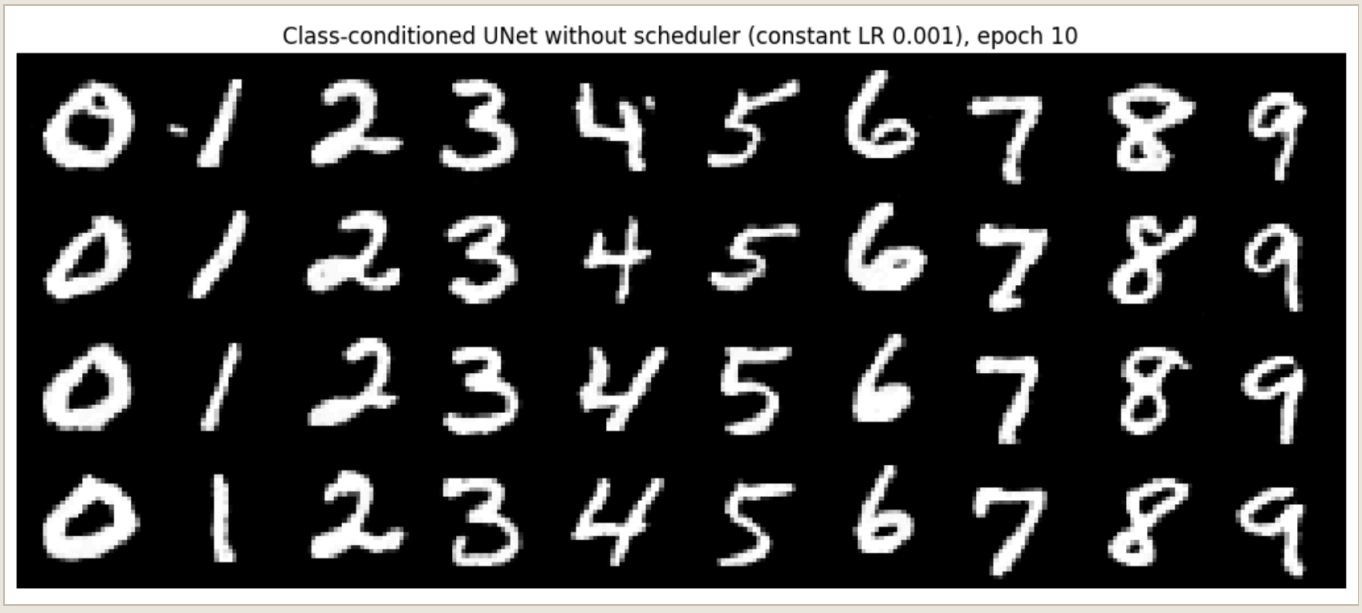


With scheduler, after epoch 5.



With scheduler, after epoch 10.

Class conditioning makes a huge difference compared to the time-only model in 2.3. Even after one epoch, every column is clearly trying to be the right digit. The 0s, 1s, 3s, 7s and 8s are already readable, but the strokes are thick and blobby, and the 2s, 4s, 5s and 9s are rough. After epoch 5 all 40 samples are the correct, legible digit with clean strokes. After epoch 10 they are a bit smoother and more consistent. The four samples in each column also differ from each other (slant, stroke width, open or closed loops), so guidance doesn't collapse each class to a single image.



Without the scheduler (constant learning rate 10^{-3}), after epoch 10.

Without the scheduler, the epoch-10 samples are just as good as with it: all 40 are the correct digit with clean strokes and similar variety. The only differences are small stray marks, such as a dot next to the first 1 and the first 4. So lowering the learning rate to a constant 10^{-3} is enough to replace the exponential schedule.